

R2R: A Relay-to-Relay Private Messaging Network

White paper and complete protocol specification

Protocol: R2R v2 (wire proto field: 1) · **Reference relay:** r2r-relay 1.0.3 (C++20) · **Reference client:** the R-2-R wallet (single-file HTML/JS) · **Document date:** 26 September 2026 (this revision: single-use client proofs in §4.4, collect from verified relays in §9.6, relay 1.0.3; voucher signature verification in §14.4, pointer and presence rules in §9.5 and §7.11, relay 1.0.2; earlier the same day: peer-table verification rules in §11, relay 1.0.1; 23 September: wallet onion routing, relay input hardening, invite activation, new seed network, onion v2 with standby relays, script key derivation)

Abstract

R2R is a private messaging network with no central server. Every message is end-to-end encrypted before it leaves the sender's device. It is carried by a mesh of small, interchangeable **relays**: dead-drop servers that hold ciphertext they cannot read, forward what belongs to other relays, and gossip a signed peer list so the network can find itself. Users have no accounts and no phone numbers. A user is an Ed25519 key pair, addressed by a hash of its public key. Relays learn only what they need to file a payload: the recipient's fingerprint. The wallet sends every stored message through onion routing by default, so the relay that files a payload for the recipient never learns who sent it, and the relay that sees the sender never learns who it is for.

The client application is called the **wallet**. It holds the user's identity key in a PIN-encrypted local vault and speaks a single JSON-over-WebSocket protocol to any relay. It keeps the user's encrypted history in an append-only journal on the relay, so a new device can restore everything from the key alone, or from a printed setup card and a PIN. Joining a relay is invite-gated. Invitation codes form a tree that grows from the relay operator's genesis codes. A new member's own codes stay locked until that member has actually used the network, and any branch of the tree can be revoked. A storage market lets users pay relays for replica space with signed USDC vouchers. The relay holds no blockchain keys, so compromising a relay never moves money.

This document describes the whole system at the byte level: identities, the wallet's local cryptography, the wire protocol, relay storage, routing, onion routing, the peer-to-peer mesh, invitations, administration and the storage market. A developer can use it to build a fully interoperable relay or client in another language. Every constant, signing string and message shape was taken from the reference source code. The test vectors in Appendix A were checked against the reference relay library and the wallet's own crypto code.

Table of contents

1. [How to read this document](#)
2. [System overview](#)
3. [Conventions and cryptographic primitives](#)
4. [Identities](#)
5. [The wallet: local keys, vault and data](#)
6. [Transport: ports, WebSocket and framing](#)
7. [Client-to-relay protocol reference](#)
8. [Relay storage](#)
9. [Routing and delivery across relays](#)
10. [Onion routing](#)
11. [The peer-to-peer relay mesh](#)
12. [Registration and the invite tree](#)
13. [Relay ownership and administration](#)
14. [The storage market and on-chain settlement](#)
15. [Wallet behaviour: messaging, groups, mail, calls, sync](#)
16. [HTTP API reference](#)
17. [Constants and configuration](#)
18. [Security and privacy analysis](#)
19. [Conformance checklist](#)
20. [Interoperability notes on the reference implementation](#)
 - [Appendix A: Test vectors](#)
 - [Appendix B: Annotated live session](#)
 - [Appendix C: Glossary](#)

1. How to read this document

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT** and **MAY** have their RFC 2119 meanings.

- **Normative:** wire formats, signing strings, key derivations, validation rules and the documented behaviour of relays. Two implementations that disagree on any of these will not interoperate.
- **Reference behaviour:** timer periods, batch sizes, retry schedules and UI-level limits. These are what the reference implementations do. Another implementation **MAY** choose differently without breaking the protocol, but matching them avoids surprises.

"Protocol v2" is the name of the current generation, as opposed to the retired v1 relays written in PHP, Python, Node and C. The integer `proto` field on the wire is still `1`. It will change only when the wire format stops being backward compatible.

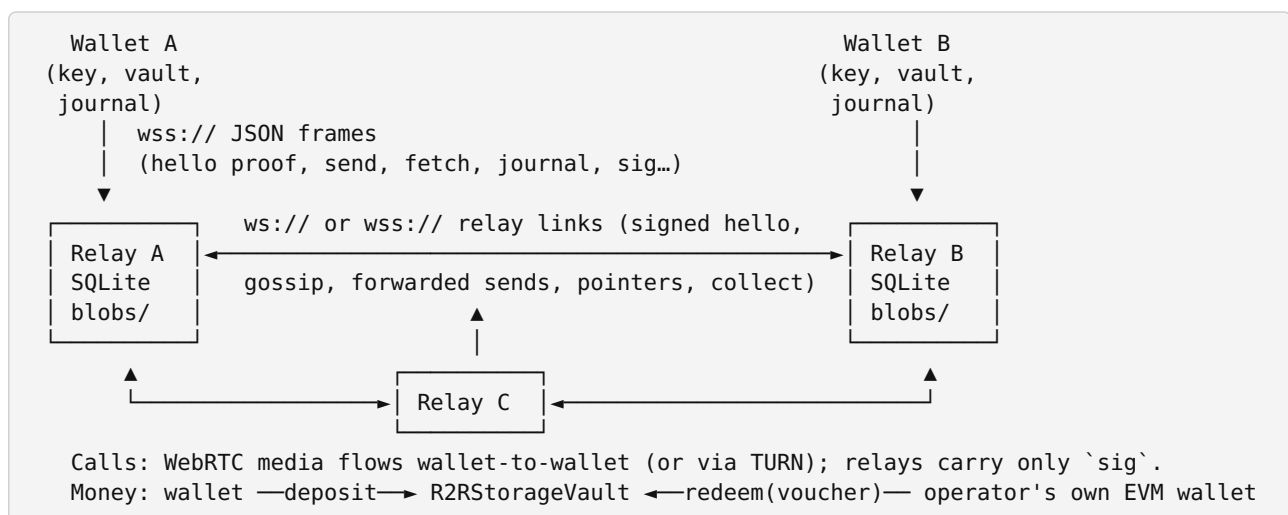
Where this document and older prose documents disagree, this document follows the code. Section 20 lists the places where the reference code has behaviour worth knowing about.

2. System overview

2.1 Roles

Role	What it is	Holds secrets?
Identity	An Ed25519 key pair. Its address is <code>fingerprint = hex(SHA-256("r2r-id-v1" pubkey))</code> .	The 32-byte seed, only on the user's devices.
Wallet	The client application. It holds the identity in a PIN-encrypted vault, encrypts and decrypts messages, and talks to relays.	Seed, PIN-derived keys, payment keys.
Relay	A server process. It stores ciphertext for fingerprints, forwards frames to other relays, gossips peers, peels onion layers and sells storage.	Only its own node seed, <code>node.key</code> . It never holds user keys.
Operator	The person running a relay host. Can mint invite codes, add owners, change quotas from the command line.	Shell access to the relay host.
Owner	An ordinary identity that the operator has marked as trusted. Can administer the relay from a wallet over signed frames.	Their own identity key.
Vault contract	<code>R2RStorageVault</code> , an ownerless EVM contract that holds user deposits and pays relays for vouchers.	None.

2.2 Architecture



2.3 Life of a message, in brief

1. Alice's wallet serialises `{"t":"text","body":"hi","rr":..., "ra":...}`. It encrypts this with `NaCl box`, using her X25519 key (converted from her Ed25519 seed) and Bob's

X25519 key (converted from his Ed25519 public key). The result is `base64(AlicePub32 || nonce24 || box)`.

- By default, Alice's wallet seals that body into an **onion** of 3 positions (entry → middle → Bob's relay; 5 with Extra privacy), each with up to 3 standby relays, and hands it to the entry relay over a fresh **anonymous** connection. Each relay peels one layer; Bob's relay files the payload in its SQLite dead drop. Without onion routing the wallet sends `{"t":"send","id":<uuid4>,"to":<Bob fp>[@relay],"body":<that base64>}` on its authenticated connection instead.
- On the direct path, the relay decides where the payload belongs: here, or at Bob's home relay. It then stores it in the dead drop or forwards it over a relay link.
- Bob's wallet, connected with a signed `hello` and subscribed with `push:true`, receives a `drop` frame. It decrypts the box, which also authenticates Alice, persists the message, and sends `ack`. The relay then deletes the payload.
- Bob's wallet sends an encrypted `{"t":"dack","ids":[...]}` payload back, and Alice's wallet shows the message as delivered.
- Both wallets append an encrypted event to their own journal on their primary relay, so the conversation can be restored on another device.

2.4 What each party learns

Party	Learns	Never learns
Relay storing a payload	Recipient fingerprint, payload size, time stored.	Plaintext, sender identity (the envelope's sender public key is inside the base64 body, which is opaque to the relay), IP addresses in any durable form.
Intermediate onion hop	The previous hop's socket, and the 1-3 candidate relays for the next position.	Sender, recipient, route length, payload.
Terminal onion hop	Recipient fingerprint.	Sender, previous hops.
Network observer on <code>ws://</code> links	Frame sizes, timing, recipient fingerprints, destination relays.	Plaintext.
Relay operator	Everything its relay stores: fingerprints, counts, sizes, invite tree.	Message content, contact graph, and, with the design as intended, anything linking a payment key to an identity.

3. Conventions and cryptographic primitives

3.1 Encodings

- Base64**: the standard alphabet (`A-Z a-z 0-9 + /`) with `=` padding, on output. Decoders MUST accept standard and URL-safe (`-`, `_`) alphabets, with or without padding. They MUST ignore embedded whitespace (`\n \r space \t`), MUST reject

data after padding and more than two `=`, and MUST reject non-zero leftover bits. The reference decoder rejects a leftover of 6 or more bits and any non-zero trailing bits.

- **Hex:** lowercase on output. Decoders accept either case. Fingerprints MUST be lowercase hex on the wire; relays lowercase and trim the ones they receive.
- **UUID v4:** `xxxxxxxx-xxxx-4xxx-Nxxx-xxxxxxxxxxxx`, where N is one of 8 9 a b (either case accepted), 36 characters. Every message id MUST be a UUID v4. Relays reject other ids with `bad_field`.
- **Time:** Unix seconds as JSON integers, unless a field is explicitly in milliseconds (some wallet-local fields are).
- **JSON:** UTF-8 objects. Key order is not significant. The reference relay emits keys in alphabetical order. A field of the wrong type MUST be treated as invalid input (see §19).
- `||` means byte concatenation. String literals in signing strings are UTF-8 bytes. `\n` is the single byte 0x0A.

3.2 Primitives

Primitive	Specification	Used for
Ed25519	RFC 8032 (seed-based keys, deterministic signatures)	User identity, relay identity, all proofs and pointer signatures
X25519	RFC 7748	Onion sealing (relay keys), NaCl box (user keys)
SHA-256	FIPS 180-4	Fingerprints, node ids, blob ids, rendezvous ranking, card-key hash
SHA3-256	FIPS 202, not Keccak	Wallet: address checksum, derived storage keys, locator seed
scrypt	RFC 7914, $N = 2^{16}$, $r = 8$, $p = 1$ (64 MiB)	Wallet: every key derived from a PIN or passphrase
Keccak-256	Original Keccak padding, as used by Ethereum	Storage market vouchers and EVM addresses only
SHA-512	FIPS 180-4	Ed25519 seed to X25519 secret conversion (via NaCl)
HKDF-SHA256	RFC 5869	Relay X25519 derivation, onion layer keys
AES-256-GCM	NIST SP 800-38D, 12-byte nonce, 16-byte tag	Onion layer encryption
NaCl <code>crypto_box</code>	X25519 + XSalsa20-Poly1305 (TweetNaCl)	End-to-end message envelopes, 1:1 attachments
NaCl <code>crypto_secretbox</code>	XSalsa20-Poly1305	Wallet vault, local store, journal entries, group and mail attachments
secp256k1 ECDSA	SEC 2; RFC 6979 nonces; low-s	Storage market payment keys and vouchers
HMAC-SHA1 / HMAC-SHA256	RFC 2104	TURN REST credentials only

Warning: SHA3-256 and Keccak-256 produce different outputs for the same input. The wallet's `sha3` helper is FIPS SHA3-256 (the `js-sha3 sha3_256` function; `SHA3-256("") = a7ffc6f8...434a`). Keccak is used only in the storage market (`keccak256("") = c5d24601...a470`). Python's `hashlib.sha3_256` is the FIPS one. Most Ethereum libraries call Keccak "sha3". Check this before anything else when a vector fails.

3.3 Domain-separation tags

Every hash, KDF and signature in R2R is domain-separated by one of these byte strings:

Tag	Construction	Section
<code>r2r-id-v1</code>	SHA-256(<code>tag ed25519_pub</code>) gives the identity fingerprint	4.1
<code>r2r-node-v1</code>	SHA-256(<code>tag node_ed25519_pub</code>)[0..16] gives the node id	4.3
<code>r2r-x25519-v1</code>	HKDF info: relay X25519 private key from <code>node.key</code>	4.3
<code>r2r-seal-v1</code>	HKDF info: onion layer key	10.2
<code>r2r-client-v1\n...</code>	Client proof-of-possession signing string	4.4
<code>r2r-hello-v1\n...</code>	Relay handshake signing string	11.2
<code>r2r-pointer-v1\n...</code>	Mail-pointer statement signed by a relay	9.5
<code>r2r-collect-v1\n...</code>	Collection authorisation signed by an identity	9.6
<code>r2r-rdv-v1</code>	Rendezvous rank SHA-256(<code>tag fp node_id</code>)	9.5
<code>r2r-voucher-v1</code>	Keccak voucher preimage prefix	14.4
<code>R2R</code>	SHA3 address checksum suffix	4.2
<code>r2r-store / r2r-journal / r2r-backup</code>	Wallet: SHA3-256(<code>seed tag</code>) gives storage keys	5.3
<code>r2r-locator / r2r-idblob</code>	Wallet: setup-card locator seed and record key	5.5
<code>turn-handle </code>	TURN per-day username handle	7.14

3.4 Relay addresses

A relay address is written `host:port`. The canonical form is produced by these rules, which every implementation MUST apply before it compares, stores or signs an address:

1. Trim whitespace. An optional scheme prefix sets the transport and the default port: `wss://` means TLS and default port 8788; `ws://` means plain and default port 8787; with no scheme the default is 8787, or 8788 if the context says TLS. Strip one trailing `/`.
2. Reject input longer than 300 bytes, any byte $\leq 0x20$ or $\geq 0x7F$, and any of `/ \ @ ? # ,` in what remains. Paths are not allowed.
3. `[v6]:port` is an IPv6 literal. A bare string containing two or more `:` is an IPv6 literal with the default port. Otherwise the form is `host[:port]`.
4. Port: 1-65535, decimal, at most 5 digits.

5. Host: 1-253 characters from `[A-Za-z0-9-._:]`, containing at least one alphanumeric character, not starting with `.` or `-`, not ending with `.`, with no `...`. Lowercase it.
6. Output `host:port`, or `[host]:port` if the host contains `:`.

Example: `WSS://Relay.Example/` becomes `relay.example:8788`, and `203.0.113.10` becomes `203.0.113.10:8787`.

3.5 Fingerprint validity

A syntactically valid fingerprint is 32-128 lowercase hex characters. Relays accept any length in that range for addressing, because the relay never checks that a fingerprint belongs to a key unless it is proving one. Identity fingerprints as produced in §4.1 are always 64 characters.

4. Identities

4.1 User identity and fingerprint

A user identity is a 32-byte random **seed**. It expands to an RFC 8032 Ed25519 key pair; NaCl's `sign.keyPair.fromSeed` gives the same result.

```
ed25519_pub = Ed25519_PublicKey(seed)           # 32 bytes
fingerprint = lowercase_hex( SHA-256( "r2r-id-v1" || ed25519_pub ) ) # 64 hex chars
```

The fingerprint is the identity's address on every relay: the mailbox name, the key for quotas and journals, and the value in `to` fields. Relays only ever see the fingerprint, plus the public key whenever the identity proves possession of it.

4.2 The shareable address (R2R_...)

Humans exchange a longer, checksummed form that carries the **public key** itself, so a contact can encrypt to it:

```
ck      = SHA3-256( ed25519_pub || "R2R" ) [0..3]      # 3-byte checksum
raw     = ed25519_pub || ck                             # 35 bytes
s       = Base32_RFC4648_alphabet_no_padding(raw)      #
"ABCDEFGH...Z34567", 56 chars
address = "R2R_" + join("_", chunks_of_8(s))
```

Encode the base32 MSB-first, 5 bits at a time. A final partial group is left-aligned (zero-padded on the right).

To decode, uppercase the input, strip an optional `R2R`, `R2R_` or `R2R-` prefix, remove every character outside `[A-Z2-7]`, decode, and require at least 35 bytes. The first 32 bytes are the key and bytes 32-34 must equal the recomputed checksum.

A contact link is `r2r:<address>[?relay=<urlencoded relay url>]`. Wallets also accept a bare 64-hex Ed25519 public key.

4.3 Relay (node) identity

A relay has exactly one secret: a 32-byte seed in `<data-dir>/node.key`, mode 0600. It is stored as 64 hex characters plus a newline; 32 raw bytes are also accepted on load. If the file is missing it is generated. Everything else derives from the seed:

```
node_ed25519_priv = seed                                     (RFC 8032 seed)
node_ed25519_pub  = Ed25519_PublicKey(seed)
node_id           = lowercase_hex( SHA-256("r2r-node-v1" || node_ed25519_pub) )[0..32] # 32
hex chars = 16 bytes
node_x25519_priv  = HKDF-SHA256(ikm = seed, salt = <empty>, info = "r2r-x25519-v1", L = 32)
node_x25519_pub   = X25519(node_x25519_priv, basepoint 9)      (standard RFC 7748
clamping applies)
```

The node id is bound to the Ed25519 key, so a relay cannot claim an id it holds no key for. The X25519 key is the onion sealing key published in `peers.json` and `node.json`.

4.4 Proof of possession (client authentication)

Binding a connection to a mailbox, claiming an invite, uploading a blob and every other authenticated action require proof that the caller holds the key behind a fingerprint. The proof is one object:

```
{ "id": "<fingerprint>", "pubkey": "<base64 ed25519 pub>", "ts": 1754331000,
  "nonce": "<16-64 chars>", "sig": "<base64 Ed25519 signature>" }
```

The signature covers exactly these bytes:

```
"r2r-client-v1\n" || fingerprint || "\n" || decimal(ts) || "\n" || nonce
```

A relay **MUST** check all of the following, in any order, and treat any failure as "not proved":

1. `id`, trimmed and lowercased, is a valid fingerprint. If not, the error code is `bad_field`.
2. `pubkey` and `sig` are present, and `nonce` is 16-64 characters long. If not, the error code is `not_authorised`.
3. $|\text{now} - \text{ts}| \leq 600$ seconds.
4. `pubkey` decodes to exactly 32 bytes and `sig` decodes.
5. `fingerprint(pubkey) == id`. This check is essential: without it, any key's valid signature would bind any mailbox.
6. `Ed25519_Verify(pubkey, signing_string, sig)`.
7. The pair (`id`, `nonce`) has not been presented before. A relay remembers every accepted proof for at least the clock window (the reference keeps them for 900 s), so a captured proof is worthless a second time, whether on a socket or in an `X-R2R-Auth` header. Wallets sign a fresh nonce for every use.

The reference wallet uses a 12-byte random nonce (24 hex characters). Before 1.0.3 the relay did not remember nonces, so a captured proof could be replayed within the ± 10 -minute window. That is acceptable because a proof only opens a session for the same key holder; it cannot be used to impersonate anyone else.

The same object carries the proof in three places:

- inside the `hello` frame (§7.1) and the `invite_claim` frame (§7.16), as top-level fields;
- in the X-R2R-Auth HTTP header, as `base64( JSON(proof) )`, where the decoded JSON must be at most 4096 bytes (§16);
- in the body of `POST /invite/claim` (§16).

5. The wallet: local keys, vault and data

This section describes the reference wallet's client-side cryptography. The formats are normative for **interoperability between wallets**: setup cards, backups, journals, and envelopes that another wallet must open. A relay never sees any of them in plaintext.

5.1 Converting Ed25519 identity keys to X25519

NaCl `box` needs X25519 keys. The wallet converts:

```
x25519_secret = clamp( SHA-512(seed)[0..32] )      # clamp: b[0] &= 248; b[31] &= 127;
b[31] |= 64
x25519_pub    = X25519(x25519_secret, 9)
              = Montgomery_u( Edwards_point(ed25519_pub) )  # u = (1 + y) / (1 - y) mod
2^255-19
```

Anyone can compute a contact's X25519 public key from their Ed25519 public key with the birational map (`libsodium crypto_sign_ed25519_pk_to_curve25519`). The wallet also announces its X25519 public key to relays in the `x25519` field of `hello` and `invite_claim`. The relay stores it (§8.1) for future sealed pointers.

5.2 The PIN-encrypted vault

The seed lives on the device only inside a vault encrypted under a key derived from the user's **PIN or passphrase** (any text; "PIN" below means either):

```
KDF(pin, salt) = scrypt(P = UTF8(pin), S = salt, N = 65536, r = 8, p = 1, dkLen = 32)
# RFC 7914. 64 MiB and ~0.4–1 s per derivation on a desktop browser, several times that
# on a phone – the same cost for every guess an attacker makes, and memory-hard, so
# GPUs and ASICs gain little over a laptop.

sbox(plaintext, key) = base64( nonce24 || crypto_secretbox(plaintext, nonce24, key) )
unsbox(s, key)       = crypto_secretbox_open(...) → plaintext or failure
```

When `sbox` is given a string, it encrypts that string's UTF-8 bytes. When given an object, it encrypts the UTF-8 bytes of `JSON.stringify(object)`.

Vault record (browser `localStorage` key `r2r1_wrap`):

```
{ "salt": "<16 random bytes, hex>", "kdf": { "alg": "scrypt", "N": 65536, "r": 8, "p": 1 },
  "wrap": "<sbox( hex(seed), KDF(pin, salt || pepper) )>", "pep": 1 }
```

`kdf` records the parameters the vault was written with, so they can be raised later. A reader **MUST** refuse parameters outside $N \in 2^{14} \dots 2^{20}$ (power of two), $r \in 1 \dots 16$, $p \in 1 \dots 4$. The reference wallet can still open a pre-release vault without `kdf` (an iterated-SHA3 key), and immediately rewraps it with `script`.

- The plaintext inside `wrap` is the **64-character lowercase hex string** of the seed, not the raw 32 bytes.
- `pepper` is empty unless the device was set up from a setup card (§5.5). In that case the 32-byte card key is stored separately under `r2r1_pepper` (hex), and `pep: 1` is set. When `pep` is absent or 0, the KDF salt is just the 16-byte salt.
- **PIN policy**, enforced on every path that sets one (new vault, restore, setup card, PIN change): at least 8 characters, not one repeated character, and, if digits only, at least 12. The wallet shows a conservative strength estimate: about 4.5 bits per character for single words, since crackers try dictionary words with substitutions first, and about 11 bits per word for phrases of 3 or more words. It converts that into how long a fast GPU would need at roughly 10 000 `script` guesses per second, and recommends a phrase of 4–5 random words.
- Unlocking means recomputing the KDF and opening `wrap`. Failure means "Wrong PIN". There is no other verifier.

Private keys are never derived from typed text. A seed is 32 random bytes, generated by the wallet or pasted as 64 hex characters, and nothing else is accepted. Hashing a memorable phrase into a key (a "brain wallet") has no secret salt and no per-user cost, so attackers crack such keys in bulk by hashing huge lists of likely phrases, however long the phrases are. A passphrase belongs in the vault KDF above, where it only unlocks a random key held on the device.

5.3 Keys derived from the seed

Key	Derivation	Protects
Store key	SHA3-256(seed "r2r-store")	Local app state, stored as <code>r2r1_data_<first 16 hex of ed25519_pub> = sbox(JSON(state), storeKey)</code>
Journal key	SHA3-256(seed "r2r-journal")	Each journal entry uploaded to relays (§15.8)
Backup key	SHA3-256(seed "r2r-backup")	Offline backups <code>R2RBK1:<sbox(JSON(state), backupKey)></code>

A backup opens only with the same seed. Restoring one creates a new vault under a new PIN and a fresh salt.

5.4 Local state (informative)

The wallet's local state object holds at least: `name`, `contacts[]` {`pub`, `addr`, `name`, `relay`, `relayAddr`}, `threads{pubHex|"g:"+gid → messages[]}`, `groups{}`, `mails[]`, `callLog[]`, `relays[]` (the **first entry is the primary relay**), `relayMeta{url → {nodeId, advertise, base}}`, `outbox[]`, `jout[]` (journal entries waiting to upload), `jseq` (the highest journal sequence number applied), `myInvites[]`, and `market {rpc, rentals[]}`. Its exact layout is a local matter. Only the journal events built from it (§15.8) cross devices.

5.5 Setup cards and locator accounts

A **setup card** lets a person restore, or first create, an identity from a printed QR code plus a PIN, without ever writing down the seed.

Card format (produced by the Setup Card Forge, or by a relay's web redemption page, §12.8):

```
R2RSC1:<cardKey: 64 hex chars = 32 random bytes>:<relayURL>[,<relayURL>...]
```

The card holds **no identity**. It holds only a random 32-byte card key (also called the pepper) and one or more relay URLs, tried in order.

Locator derivation:

```
K          = KDF(pin, cardKeyBytes)                # the §5.2 scrypt KDF, salt = the
32-byte card key
locatorSeed = SHA3-256( K || "r2r-locator" )        # an Ed25519 seed
recordKey   = SHA3-256( K || "r2r-idblob" )         # a secretbox key
locator     = Ed25519 key pair from locatorSeed; its fingerprint is an ordinary identity
fingerprint
```

The locator is an ordinary relay identity. It proves itself with a normal `hello` and needs no registration or invite. Its **journal** on each card relay holds the identity record:

Record (inside <code>sbox(..., recordKey)</code>)	Meaning
<code>{"s": "<64-hex seed>"}</code>	This card plus this PIN restore this seed.
<code>{"r": 1}</code>	Revoked. The PIN was changed; ignore every earlier <code>s</code> record.

Setup-from-card algorithm (reference):

1. Parse the card and require a PIN of at least 8 characters. Derive the locator.
2. For each card relay, in order: open a WebSocket, send the locator's `hello`, and read its whole journal (`journal_read` from 0, following `more`). Relays that fail are skipped.
3. For each reachable relay, scan its records in order: a decryptable `{s}` sets `found = seed`, and `{r:1}` sets `found = null`. The last record wins.
4. `seed` is the first relay's `found`. Relays that were reachable but had no seed are "missing".
5. If no relay had a seed, this is a **new user**: generate a random seed.
6. Append `sbox({"s": hex(seed)}, recordKey)` to every missing relay's locator journal. This heals partial copies. If the seed is new and no relay accepted the record, fail.
7. Create the local vault (§5.2) with `salt16 || cardKey` as the KDF salt. Store `r2r1_pepper` and the card relay list, and use the card relays as the relay list.

The wrong-PIN invariant. A wrong PIN derives a different locator, which has an empty journal, so the wallet silently creates a fresh identity. An attacker holding the card cannot tell whether the PIN they guessed is someone's real PIN. Offline guessing is still possible for anyone who has the card key and can query the relays, which is why the PIN must be long (§18).

Change PIN. Verify the old PIN against the vault. Then, for every card relay, append `{s:seed}` under the **new** locator. If any relay fails, abort and leave the PIN unchanged. Only after every relay confirms, append `{r:1}` under the **old** locator (best effort), and rewrap the local vault.

Moving relays. Copy every journal entry, verbatim, from the old primary to the new one with `journal_read` and `journal_append`. Then ensure the new relay's locator journal contains the `{s}` record. The Forge can reprint a card with the same card key and a new relay list.

5.6 Other wallet string formats

Prefix	Format	Purpose
r2r:	r2r:<R2R_address>[?relay=<url>]	Contact invitation (QR or link)
R2REI1:	R2REI1:<16-byte salt hex>:<sbox(contactLink, KDF(pin, salt))> (script KDF; PIN ≥ 6 characters, two words recommended)	PIN-protected contact invitation
R2RBK1:	R2RBK1:<sbox(JSON(state), backupKey)>	Full local backup
R2RSC1:	See §5.5	Setup card
R2R-...	See §12.2	Relay invite code (registration)

5.7 The message envelope

Every message, receipt, group operation and call signal between users is one envelope:

```
body = base64( sender_ed25519_pub[32] || nonce[24] || crypto_box( UTF8(JSON(payload)), nonce,
                                                                recipient_x25519_pub,
sender_x25519_secret ) )
```

- `nonce` is 24 random bytes. The minimum valid length is $32 + 24 + 16$ (Poly1305 tag) + $1 = 57$ bytes.
- The **recipient authenticates the sender** by opening the box: it converts the sender public key in the prefix to X25519 and runs `box_open` with its own X25519 secret. A successful open proves the holder of that Ed25519 key's X25519 twin wrote the payload. Relays neither see nor verify the sender, so the `from` is established end to end and never by a relay.
- The recipient **MUST** discard envelopes that do not open or whose plaintext is not a JSON object.
- The payload `t` values are listed in §15. The wallet adds `rr` (its primary relay URL) and `ra` (that relay's advertised `host:port`) to every payload, so the recipient learns where to reply.

6. Transport: ports, WebSocket and framing

6.1 Listeners

A relay process serves two ports with **identical** application logic:

Default port	Scheme	Typical use
8787	ws:// and http://	Relay-to-relay links; local and trusted clients
8788	wss:// and https://	Remote clients and browsers

TLS is TLS 1.2 or later. If the certificate or key cannot be read, the relay logs a warning and serves `ws://` only. With `--no-tls` the TLS listener is not opened. A relay behind a TLS-terminating proxy is told the public URL with `--public-ws-url`. HTTP requests carrying `X-Forwarded-Proto: https` are treated as secure when the relay renders links.

The bridge. The outgoing hop's scheme is a property of the next peer, not of how a frame arrived. A frame received over `wss://` MAY leave over `ws://`. Routing metadata on plaintext hops is visible to network observers; payloads never are.

6.2 WebSocket

- Any HTTP/1.1 request with a WebSocket upgrade, **on any path**, is upgraded. Wallets use `<relay-url>/ws` and relays dial `/r2r`. A relay MUST NOT depend on the path.
- The subprotocol is optional. If the client offers `r2r.v1` in `Sec-WebSocket-Protocol`, the relay echoes it.
- Text frames only.** A binary frame closes the connection with code 1003 (`unknown_data`, reason "text frames only").
- Maximum message size: `max_frame` (default 1 MiB). With `--max-payload N` it becomes `2N + 65536`. Larger messages fail at the WebSocket layer, or are answered with `err too_big`.
- Idle timeout is 180 s, with WebSocket keep-alive pings. The wallet additionally sends a `{"t":"ping"}` frame every 25 s.
- Each connection has an outbound queue of 256 messages. A client that does not drain it is disconnected with code 1013 (`try_again_later`).

6.3 Frames

Every message is one JSON object with a string field `t`, the frame type. Binary data travels inside it as base64. Unknown extra fields MUST be ignored.

Errors always have this shape:

```
{ "t": "err", "code": "<code>", "msg": "<human text>", "ref": "<optional: message id or frame type>" }
```

Code	Meaning
<code>bad_frame</code>	Not a JSON object, or <code>t</code> is missing

Code	Meaning
<code>bad_field</code>	A field is missing, malformed or out of range
<code>too_big</code>	Frame or payload over the limit
<code>rate_limited</code>	Slow down
<code>not_authorised</code>	Proof missing or invalid, not an owner, wrong role
<code>unknown_type</code>	Unsupported <code>t</code> (<code>ref</code> holds the type)
<code>no_route</code>	Cannot reach the next hop, or the onion layer is not addressed here
<code>quota</code>	Storage allowance or mailbox full
<code>invite_invalid</code> / <code>invite_used</code> / <code>invite_revoked</code> / <code>invite_locked</code> / <code>already_registered</code>	Invite claim failures
<code>market_off</code> / <code>market_full</code>	Storage market disabled or out of inventory
<code>not_online</code>	<code>sig</code> target is not connected
<code>hop_limit</code>	Forwarding hop budget exhausted
<code>internal</code>	Server-side failure

Rule: an error is terminal. A relay MUST NOT answer an `err` frame, and MUST silently ignore reply-type frames it did not ask for: `sent` `drop` `fetch_done` `invite_ok` `mail` `ack_ok` `located` `mail_at` `deposit_ok` `journal` `journal_ok` `rent_ok` `voucher_ok` `admin_ok`. Two relays that each answered the other's reply with "I don't recognise that" would saturate the link. That failure happened during development, with 2.4 million frames exchanged between two idle relays.

Rate limiting (reference). Each connection has an inbound token bucket of 40 frames/s with a burst of 120. A throttled frame is dropped and answered with `rate_limited`. After 200 consecutive throttled frames the connection is closed with code 1013. Outbound `err` frames are themselves limited to 2/s with a burst of 6; errors beyond that are dropped silently.

6.4 Session state

Each connection starts **anonymous**. It becomes a:

- **client session with identity**, after a `hello` whose proof verifies (§7.1);
- **anonymous client**, after a `hello` without an `id`. It may `send`, `onion`, `ping`, `ice` and `watch`, but not read anyone's mail;
- **relay session**, after a verified relay `hello` or `welcome` (§11.2).

A `hello` with an unprovable `id` gets an `err` and leaves the session unbound. The socket stays open.

Frames that need no identity (`send`, `onion`, `ping`, `ice`, `watch`, `invite_claim`) are accepted even before any `hello`.

7. Client-to-relay protocol reference

Each entry gives the request, the reply, and the rules the relay applies. "Requires identity" means the session must have completed a proved `hello` (§4.4); otherwise the reply is `err not_authorised` ("send a hello with your id first").

7.1 `hello` → `welcome`

```
{ "t":"hello", "id":"<fp>", "pubkey":"<b64>", "ts":1754331000, "nonce":"<hex>",
  "sig":"<b64>",
  "x25519":"<b64 32-byte curve25519 pub, optional>" }
```

`role` defaults to `"client"`; relays send `"role":"relay"` (§11.2). If `id` is absent or empty, the session becomes an anonymous client. If `id` is present, the proof **MUST** verify, or the relay replies `err` and sends no `welcome`. On success the relay binds the session to the fingerprint (several sessions per fingerprint are allowed), updates `identities.last_seen` rounded down to the hour, stores `x25519` if it decodes to 32 bytes and the identity is registered, pushes `presence` to watchers, and replies:

```
{ "t":"welcome", "proto":1, "role":"relay", "version":"1.0.0",
  "node_id":"<32 hex>", "advertise":"<host:port or empty>", "ed25519":"<b64>",
  "x25519":"<b64>",
  "tls":true, "ws_port":8787, "wss_port":8788, "ts":<unix>, "nonce":"<24 hex>",
  "sig":"<b64>",
  "you":"<fp or empty>", "pending":<payloads waiting>, "ttl_days":7, "max_payload":262144,
  "peers":<verified peer count>, "push":true,
  "quota_bytes":<allowance>, "used_bytes":<usage>, "owner":<bool>,           // only when
  identified
  "market":{"price_gb_epoch_micro":500000,"available_bytes":<n>} }         // only when the
  market is on
```

The first block is the relay's signed identity frame (§11.2). A client **MAY** verify `sig` to authenticate the relay's node key. `role:"relay"` in a `welcome` sent to a client describes the relay, not the client. `push:true` advertises support for live delivery.

7.2 `send` → `sent`

```
{ "t":"send", "id":"<uuid4, optional>", "to":"<fp> | <fp>@<relay>", "body":"<b64
  ciphertext>",
  "hint":"<opaque ≤128 chars, optional>", "hops":<int, relays only> }
```

Does **not** require identity. Anyone may leave a payload for any fingerprint, and only the key holder can collect it. The relay:

1. Uses `id` if present and generates a UUID v4 otherwise. A non-UUID-v4 id is rejected with `bad_field`.
2. Parses `to` (§9.1) and fails with `bad_field` if it is invalid. Decodes `body`; empty or invalid base64 fails with `bad_field`, and more than `max_payload` (256 KiB) decoded bytes fails with `too_big`, `ref = id`.
3. Drops `hint` if it is longer than 128 characters.

4. Deduplicates. If the message id was seen in the last 15 minutes, it replies `sent {status:"duplicate"}` and stops.
5. Chooses a destination and stores or forwards the payload as described in §9.2.

Replies go only to client sessions; relays get no reply to a forwarded `send`:

```
{ "t":"sent", "id":"<uuid>", "status":"stored",    "expires_at":<unix> }
{ "t":"sent", "id":"<uuid>", "status":"forwarded", "via":"<host:port>" }
{ "t":"sent", "id":"<uuid>", "status":"duplicate" }
```

Errors: `quota` ("recipient mailbox is full", `ref = id`), `hop_limit`, `internal`.

`sent` confirms only that a relay accepted responsibility for the payload. It is not a delivery receipt; see §15.3 for those.

7.3 `subscribe` → `mail`, and `live drop` pushes

```
{ "t":"subscribe", "push":true }
```

Requires identity. The relay marks the session subscribed and remembers `push` (default `false`), then replies `{"t":"mail", "pending":<n>}`.

Whenever a payload is stored for this fingerprint:

- every session with `push:true` receives the full `drop` frame immediately, in the same shape as a `fetch` reply and with its `seq`;
- then every subscribed session receives `{"t":"mail", "pending":<n>}`.

A pushed payload stays stored until it is **acked**, so a push that is lost is redelivered by the next `fetch`.

7.4 `fetch` → `drop`... `fetch_done`

```
{ "t":"fetch", "since":0, "max":64 }
```

Requires identity. `since` is an **exclusive** sequence cursor (default 0). `max` defaults to 64 and is clamped to 1-128. The relay streams unexpired drops for this fingerprint in ascending `seq` order:

```
{ "t":"drop", "id":"<uuid>", "seq":<int>, "body":"<b64>", "hint":"<if any>",
  "created_at":<unix>, "expires_at":<unix> }
```

followed by:

```
{ "t":"fetch_done", "count":<n>, "cursor":<last seq or since>, "more":<count == max> }
```

`seq` is the only safe cursor. Timestamps are not, because two payloads stored in the same second share one. Aacked payloads are deleted, so a client that acks everything can always fetch with `since: 0`. The reference wallet does exactly that.

7.5 `ack` → `ack_ok`

```
{ "t":"ack", "ids":["<uuid>", ...] }
```

Requires identity. Up to 256 valid UUIDs are processed per frame; other entries are ignored. Payloads addressed to this fingerprint with those ids are deleted in one transaction. The reply is `{ "t": "ack_ok", "removed": <n> }`. Wallets **MUST** ack only after they have decrypted and **persisted** the message, or decided to discard it.

7.6 `locate` → `located`, and `live mail_at`

```
{ "t":"locate" }
→ { "t":"located", "pending":<local count>, "pointers":
  [ { "address": "<host:port>", "count":<n>, "ts":<unix> } ] }
```

Requires identity. Lists other relays that have signed statements saying they hold mail for this fingerprint (§9.5). When a new pointer arrives while the identity is connected and subscribed, the relay pushes `{ "t": "mail_at", "address": "<host:port>", "count": <n> }`.

7.7 `deposit` → `deposit_ok` (gather mail from other relays)

```
{ "t":"deposit", "id": "<own fp>", "pubkey": "<b64 ed25519>", "scope": "collect" | "collect-delete",
  "targets": [ { "address": "<holder host:port>", "ts": <unix>, "nonce": "<16-64 chars>",
    "sig": "<b64>" } ] }
```

Requires identity, and `id` **MUST** equal the session's fingerprint ("deposit only your own mail"). `targets` must hold 1–16 entries. Each target is a **collection authorisation** signed by the identity over:

```
"r2r-collect-v1\n" || fp || "\n" || holder_address || "\n" || scope || "\n" || decimal(ts) || "\n" || nonce
```

where `holder_address` is the holder's canonical address exactly as it advertises itself. The relay forwards each authorisation to its holder (§9.6) and replies `{ "t": "deposit_ok", "targets": <number routed> }`. Collected payloads then arrive through the normal store path, so they appear as `drop` pushes and `mail` notifications.

7.8 `journal_append` → `journal_ok`

```
{ "t":"journal_append", "data": "<b64, ≤ 640 KiB decoded>" }
→ { "t":"journal_ok", "seq":<int> }
```

Requires identity. Appends an opaque entry to the identity's journal. `seq` is strictly increasing per identity: `MAX(seq)+1`, starting at 1. The entry counts against the storage allowance, and an append over the allowance fails with `err quota`. An empty or invalid `data` is rejected with `bad_field`, and one over 640 KiB with `too_big`.

7.9 journal_read → journal

```
{ "t":"journal_read", "since":0, "max":200 }
→ { "t":"journal", "entries":[ { "seq":<n>,"data":<b64>,"created_at":<unix>} ],
    "cursor":<last seq or since>, "more":<entries == max> }
```

Requires identity. `since` is exclusive. `max` defaults to 200 and is clamped to 1–200. A reply is also cut before the entry that would push its total over 2 MiB, but it always contains at least one entry when any exist. `more` is `true` whenever the page stopped early, whether because it reached `max` or because of the byte cap. Journal entries **do not expire**.

7.10 quota → quota

```
{ "t":"quota" }
→ { "t":"quota", "quota_bytes":<allowance>, "used_bytes":<drops+journal+blobs>,
    "used_payloads":<n>,
    "max_payloads":4096, "custom":<has an operator-set quota> }
```

Requires identity.

7.11 watch → presence, and presence from the client

```
{ "t":"watch", "ids":["<fp>", ...] } // replaces the session's watch list; at most
512 are kept
→ { "t":"presence", "id":<fp>, "state":"online"|"away"|"offline", "seen":<unix or
0> } // one per id, now
{ "t":"presence", "state":"away" } // client sets its own state; any other value
means "online"
```

`watch` requires a session that proved an identity (not `authorised` otherwise): `presence` is not an anonymous oracle. Presence covers only identities connected **to this relay**. Watchers are pushed a new `presence` whenever a watched identity connects, disconnects or changes state. If any session of an identity is online, that identity is `online`. `seen` is the time the identity was last connected or last proved itself, held in memory only and never persisted.

7.12 sig (call signalling)

```
{ "t":"sig", "to":<fp> | <fp>@<relay>, "body":<opaque, ≤ max_payload chars>,
"call":<optional string> }
→ delivered as { "t":"sig", "from":<sender fp>, "body":<...>, "call":<...> }
```

Requires identity for clients. The relay sets `from` to the session's proved fingerprint. The target is resolved like a `send` (§9.2):

- **Local:** the frame is delivered to **every** live session of the target. If none is connected, the reply is `err not_online`. Signalling is never stored.
- **Remote:** the frame is forwarded as `{t:"sig", from, to:"fp@dest", body, call?, hops}`. If there is no route, the reply is `err no_route`.

A relay receiving `sig` over a relay link trusts that link's `from`. Wallets therefore put an **authenticated envelope** (§5.7) in `body` and MUST authenticate the sender from the envelope, not from `from`.

7.13 onion

```
{ "t":"onion", "blob":"<b64 sealed layer>", "hops":<int, optional> }
```

Does not require identity. On success the relay stays **silent**, and only failures produce an `err`. The format and processing rules are in §10.

7.14 ice → ice

```
{ "t":"ice" } → { "t":"ice", "iceServers":[ { "urls":"stun:..."},
{ "urls":"turn:...","username":"...","credential":"..." } ], "ttl":600 }
```

`iceServers` always holds the STUN URL (default `stun:stun.l.google.com:19302`). A TURN entry is added when one is configured:

- **Static credentials:** the configured `username` and `credential`.
- **Shared secret** (coturn `use-auth-secret`, the preferred mode):

```
username  = decimal(now + ttl) [ ":" + hex( HMAC-SHA256(secret, "turn-handle|" ||
decimal(now / 86400) || "|" || fp) )[0..12] ]
credential = base64( HMAC-SHA1(secret, username) )
```

The `:handle` suffix appears only when the session has an identity. It rotates daily and does not identify the user without the secret. Clients should request fresh credentials before `ttl` expires. `GET /ice` (§16) returns the same data.

7.15 ping → pong, and bye

`{ "t":"ping", "nonce":"..." }` is answered with `{ "t":"pong", "time":<unix>, "nonce":"..." }`.
`{ "t":"bye" }` makes the relay close the connection with code 1000.

7.16 invite_claim → invite_ok

```
{ "t":"invite_claim", "code":"R2R-...", "id":"<fp>", "pubkey":"<b64>", "ts":..., "nonce":"...",
"sig":"...",
"home_relay":"<host:port, optional>", "x25519":"<b64, optional>" }
→ { "t":"invite_ok", "invites":["R2R-...", "R2R-...", "R2R-..."], "locked":true,
"home_relay":"<this relay's advertise>", "node_id":"<32 hex>",
"unlock":{"received_needed":3, "vouch_needed":<bool>},
"inviter":{"id":"<fp>", "pubkey":"<b64 ed25519>"} } // only when an identity
issued the code
```

This frame is self-contained: it carries its own proof and does not bind the session. The semantics and error codes (`invite_invalid`, `invite_used`, `invite_revoked`, `invite_locked`, `already_registered`) are in §12.4. The reference wallet sends it as the **first** frame on a fresh connection and closes the connection after the reply.

7.17 rent, voucher

See §14.

7.18 vouch → vouch_ok

```
{ "t":"vouch", "id":"<fp of a member this identity invited>" }
→ { "t":"vouch_ok", "id":"<fp>", "active":<the member's codes are now unlocked> }
```

Requires identity. It is accepted only when the session's fingerprint is the `issued_by` of the code the member claimed on this relay; otherwise the reply is `err not_authorized`.

Vouching twice is harmless. See §12.5.

7.19 invite_status → invite_status

```
{ "t":"invite_status" }
→ { "t":"invite_status", "registered":<bool>, "active":<bool>, "received":<n>,
    "received_needed":3,
    "vouch_needed":<bool>, "vouched":<bool>, "inviter":{"id","pubkey"}?,
    "invites":[ {"code":"R2R-...", "state":"locked"|"open"|"burned"|"revoked"} ] }
```

Requires identity. It lists every code this identity issued on this relay, and its progress towards activation (§12.5).

7.20 Owner frames (admin_*)

See §13.

8. Relay storage

The reference relay keeps all state in one SQLite database (`<data-dir>/r2r.db`) plus a content-addressed blob directory. Other languages MAY use any store, provided they keep the **observable semantics** described here: transactional invite burns, monotonic sequence numbers, quota arithmetic, expiry, and deletion on ack.

SQLite settings: `journal_mode=WAL`, `synchronous=NORMAL`, `foreign_keys=ON`, `temp_store=MEMORY`, `cache_size=-8000`, and `secure_delete=ON`, so freed pages holding ciphertext are overwritten. On SIGTERM the relay checkpoints the WAL (`TRUNCATE`) and closes the database cleanly. The schema version (`PRAGMA user_version`) is 5. Migrations check for column presence, so an interrupted upgrade can resume.

8.1 Schema

```
-- Dead drops: ciphertext waiting for a recipient. rowid is the fetch cursor ("seq").
CREATE TABLE drops(
  id TEXT PRIMARY KEY,           -- message UUID (dedup key)
  recipient TEXT NOT NULL,       -- fingerprint
  from_hint TEXT NOT NULL DEFAULT '', -- opaque sender token, never verified
  payload BLOB NOT NULL,
  created_at INTEGER NOT NULL,
  expires_at INTEGER NOT NULL);
```

```

CREATE INDEX idx_drops_recipient ON drops(recipient, created_at);
CREATE INDEX idx_drops_expiry ON drops(expires_at);

-- Invite codes (§12). Kept forever once burned or revoked, so they can never be replayed.
CREATE TABLE invites(
  code TEXT PRIMARY KEY, created_at INTEGER NOT NULL, issued_by TEXT NOT NULL DEFAULT '',
  burned_at INTEGER, burned_by TEXT,
  revoked_at INTEGER, revoked_by TEXT,
  parent_code TEXT,          -- the claimed code that minted this one (lineage)
  contact_token TEXT,        -- 32 hex chars; /m/<token> resolves to issued_by
  locked INTEGER NOT NULL DEFAULT 0) WITHOUT ROWID;  -- 1 until the issuing member is
active
CREATE INDEX idx_invites_open ON invites(burned_at);
CREATE INDEX idx_invites_parent ON invites(parent_code);
CREATE INDEX idx_invites_token ON invites(contact_token);

-- Registered identities: where their mail lives. Never gossiped.
CREATE TABLE identities(
  fingerprint TEXT PRIMARY KEY, home_relay TEXT NOT NULL, pubkey TEXT NOT NULL DEFAULT '',
  registered_at INTEGER NOT NULL, last_seen INTEGER NOT NULL,  -- last_seen rounded down to
the hour
  x25519 TEXT NOT NULL DEFAULT '',
  received INTEGER NOT NULL DEFAULT 0,  -- payloads received and acked since registering
  vouched_at INTEGER) WITHOUT ROWID;  -- when the inviter vouched (NULL = not yet)

CREATE TABLE journal(fingerprint TEXT NOT NULL, seq INTEGER NOT NULL, data BLOB NOT NULL,
  created_at INTEGER NOT NULL, PRIMARY KEY(fingerprint, seq));

-- One row per (blob, owner): content stored once, accounted per owner.
CREATE TABLE blobs(id TEXT NOT NULL, owner TEXT NOT NULL, size INTEGER NOT NULL,
  created_at INTEGER NOT NULL, expires_at INTEGER NOT NULL, PRIMARY KEY(id, owner)) WITHOUT
ROWID;

CREATE TABLE owners(fingerprint TEXT PRIMARY KEY, added_at INTEGER NOT NULL) WITHOUT ROWID;
CREATE TABLE quotas(fingerprint TEXT PRIMARY KEY, max_bytes INTEGER NOT NULL, updated_at
INTEGER NOT NULL) WITHOUT ROWID;
CREATE TABLE account_ttls(fingerprint TEXT PRIMARY KEY, ttl_days INTEGER NOT NULL,
updated_at INTEGER NOT NULL) WITHOUT ROWID;
CREATE TABLE cards(card_key_hash TEXT PRIMARY KEY, invite_code TEXT NOT NULL, created_at
INTEGER NOT NULL) WITHOUT ROWID;

-- Signed statements "relay <address> holds <count> payloads for <fingerprint>".
CREATE TABLE pointers(fingerprint TEXT NOT NULL, address TEXT NOT NULL, count INTEGER NOT
NULL,
  node_id TEXT NOT NULL, ed25519 TEXT NOT NULL, sig TEXT NOT NULL,
  created_at INTEGER NOT NULL, expires_at INTEGER NOT NULL, PRIMARY KEY(fingerprint,
address)) WITHOUT ROWID;

CREATE TABLE used_authz(nonce TEXT PRIMARY KEY, created_at INTEGER NOT NULL) WITHOUT ROWID;

CREATE TABLE rentals(id TEXT PRIMARY KEY, fingerprint TEXT NOT NULL, bytes INTEGER NOT NULL,
  price_micro INTEGER NOT NULL, payment_key TEXT NOT NULL DEFAULT '', created_at INTEGER NOT
NULL,
  paid_until INTEGER NOT NULL, state TEXT NOT NULL DEFAULT 'active') WITHOUT ROWID;  --
active|lapsed|closed
CREATE TABLE vouchers(payment_key TEXT NOT NULL, payout TEXT NOT NULL, cumulative_micro
INTEGER NOT NULL,
  sig TEXT NOT NULL, raw TEXT NOT NULL, created_at INTEGER NOT NULL,
  PRIMARY KEY(payment_key, payout, cumulative_micro));

CREATE TABLE meta(k TEXT PRIMARY KEY, v TEXT NOT NULL) WITHOUT ROWID;  -- e.g.
advertise_override

```

What is never stored: IP addresses, ports, user agents, client hostnames, plaintext, contact graphs, and per-message sender identities.

8.2 Dead drops

Storing a payload for recipient *R* (the reference `store_locally`):

1. `expires_at = now + ttl_days(R) × 86400`. `ttl_days(R)` is the per-account override if one exists, otherwise the relay default of 7 days. An override of `-1` means "keep forever" and becomes 36 500 days.
2. **Mailbox check:** count and total bytes of *R*'s unexpired drops. Reject with `quota` if `count ≥ 4096`, or if `drop_bytes + len > allowance(R)` (§8.5).
3. **Common pool check** (§8.6).
4. `INSERT OR IGNORE`. If the id already exists the result is `duplicate`. Otherwise the new row's rowid is its `seq`.
5. On success: reply `sent stored` to the originating client (if any), push the `drop` to `push` sessions of *R*, send `mail` to subscribed sessions of *R*, and **emit a mail pointer** if *R* is not homed here and not connected here (§9.5).

`seq` values come from one rowid space shared by all recipients. They are strictly increasing per relay, but not contiguous per recipient.

Deletion happens when the recipient acks, when a signed `collect-delete` hands the payload over (§9.6), when an owner deletes the account, or when the sweep removes it after expiry.

8.3 Journal

An append-only, per-identity log of opaque entries, keyed `(fingerprint, seq)`, with `seq = MAX(seq)+1` assigned inside a `BEGIN IMMEDIATE` transaction together with the allowance check. Entries have no expiry and are removed only by account deletion. The relay cannot read them. §15.8 describes the wallet's event format.

8.4 Blobs (voice, video, large files)

Blobs travel over HTTP (§16) because they are much larger than a frame.

- **Id:** `lowercase_hex(SHA-256(content))`, 64 characters. The same bytes uploaded by five people are stored once, and a downloader can verify what it received.
- **Path:** `<blob-dir>/<id[0..2]>/<id>`. Directories are sharded by the first two hex characters. Files are written atomically (`temp file`, `fsync`, `rename`, `fsync` of the directory) with mode `0600`.
- **References:** a row per `(id, owner)`, with `expires_at = now + ttl_days(owner) × 86400`. A second upload of the same content by the same owner is `duplicate:true`. Its expiry is **not** extended.
- **Allowance:** the upload is refused with HTTP 413 `quota` if `used(owner) + size > allowance(owner)` or the common pool would overflow. The bytes are then left for the sweep to remove if nothing references them.

- **Download** requires any proved identity: anyone who holds the id may fetch it. The id travels inside an end-to-end encrypted message, and the content is ciphertext only the intended recipients can open.
- **GC**: the sweep deletes expired reference rows. It deletes a file only when **no** reference to that id remains.

8.5 Allowances

```
allowance(fp) = ( quotas.max_bytes if a row exists, else default_quota_mb × 1 MiB )
                + SUM(rentals.bytes WHERE fingerprint = fp AND state = 'active' AND paid_until
> now)
used(fp)       = SUM(LENGTH(drops.payload) WHERE recipient = fp)
                + SUM(LENGTH(journal.data)  WHERE fingerprint = fp)
                + SUM(blobs.size             WHERE owner = fp)
```

The default allowance is **1 MiB** (`--default-quota-mb`). The operator raises individual allowances with `--set-quota`, or an owner raises them with `admin_set_quota` (§13). Going over the limit refuses new data; **nothing already stored is deleted**.

The three store paths do not check exactly the same thing. The dead-drop path compares only the recipient's **unexpired drop bytes** (plus the 4096-drop cap) against the allowance. The journal and blob paths compare **total** `used(fp)` against the allowance. Paid rentals raise every path.

8.6 Storage pools

The relay's disk is split into budgets. Each is 0 by default, meaning unlimited or disabled:

Pool	Flag	Enforcement
Common	<code>--pool-common-mb</code>	Caps the total bytes held for all free-tier identities, which are those with neither a custom quota nor an active paid rental. Every store path checks <code>free_tier_bytes + len ≤ cap</code> . The running total is memoised, refreshed every 30 s, and increased on each accepted store.
Market	<code>--pool-market-mb</code>	The inventory of the storage market (§14). The sum of active rentals can never exceed it.
Personal	<code>--pool-personal-mb</code>	Declared for the operator's own use and published in <code>/status.json</code> ; not enforced.

8.7 The sweep

Once 60 s after start, then every 15 minutes, the relay:

1. deletes drops with `expires_at ≤ now`;
2. deletes expired blob references, then deletes the files of ids that no longer have any reference;
3. deletes expired pointers;
4. deletes `used_authz` nonces older than 8 days;
5. marks rentals with `paid_until < now` as `lapsed`. Their allowance stops at once, and their data then decays under the normal expiry rules. Nothing is deleted as a penalty.

8.8 Retention of metadata

- `identities.last_seen` is rounded down to the hour.
- Presence `seen` is kept in memory only.
- Logs never contain client IP addresses, payloads, full fingerprints (only 8-character prefixes) or invite codes. Inbound connections are named by a random 12-hex id that disappears when the connection closes. Peer relay addresses are logged, because they are public.

9. Routing and delivery across relays

9.1 Addressing

A recipient (`to`) is either:

<code><fingerprint></code>	# bare: "wherever this identity lives"
<code><fingerprint>@<relay></code>	# qualified: "on that relay"

`<relay>` may be `host:port`, `host` (port 8787), `ws://host[:port]` or `wss://host[:port]` (port 8788). It is canonicalised with §3.4, and the fingerprint is trimmed and lowercased. The whole target must be at most 400 characters.

9.2 Choosing a destination (`send` and `sig`)

```

destination = target.relay                # explicit @relay wins
if destination is empty:
    destination = identities.home_relay(target.fp) # registered home, if this relay knows
one
local = destination is empty OR destination == self_advertise
if local:
    store_locally(...)                    # send; for sig: deliver to live
sessions or err not_online
else:
    hops = frame.hops (default 8); if hops ≤ 0: err hop_limit
    forward { t:"send", id, to:"<fp>@<destination>", body, hint?, hops: hops-1 } to
destination
    if the frame cannot even be queued: store_locally(...) # keep it here rather than lose
it
    reply sent{status:"forwarded", via:destination}

```

Home relays are recorded when an invite is claimed (§12.4) and are **not gossiped**. The network deliberately has no global directory of who lives where. A relay therefore knows the home only of identities registered on it. For anyone else, a bare-fingerprint send is stored locally, and the recipient finds it through pointers (§9.5). Wallets learn their correspondents' relays from the `rr` and `ra` fields inside envelopes (§5.7), and address them explicitly.

9.3 Deduplication and loop prevention

- A **seen cache** of message keys lives 15 minutes and holds at most 200 000 entries. It has three namespaces: `m:<message id>` for `send`, `o:<first 32 hex of SHA-256(blob)>` for onion layers, and `p:<fp>|<address>|<ts>` for pointers.
- The **drops.id primary key** deduplicates at storage time, with no time limit.
- A **hop budget** caps forwarding: `hops` defaults to 8 (`--max-hops`), each forward decrements it, and a frame arriving at 0 is refused with `hop_limit`. Pointers use their own budget of 4.

9.4 Links, the pending queue and store-on-failure

`route_to_relay(address, frame)` sends over an existing link to that address, **whichever side dialled it**, if one exists. Otherwise the frame is queued and the peer is dialled:

- The queue holds at most 64 frames or 4 MiB per peer. Frames beyond that are dropped with a log warning.
- Queued frames are flushed when a dial succeeds, or when an **inbound** link from that peer appears. Every 15 s the maintenance timer re-dials peers that have frames waiting, regardless of the outbound-link budget.
- After **300 s** without delivery, a queued `send` is **parked in the local dead drop** of the relay that queued it, so the recipient can still collect it, and a pointer is emitted. Other frame types (onion layers, pointers, control frames) are discarded, because an onion layer is sealed to the unreachable hop and cannot be stored for anyone.

A relay introduced only through a `to` address is added to the peer table as a plain `ws://` peer.

9.5 Mail pointers: finding mail parked elsewhere

When a relay stores a payload for an identity that **it does not home and that is not connected to it**, and the relay has an advertise address, it emits a signed **pointer**:

```
statement = "r2r-pointer-v1\n" || fp || "\n" || self_advertise || "\n" || decimal(ts)
sig       = Ed25519_Sign(node key, statement)
frame     = { "t":"pointer", "fp", "address":self_advertise, "count":<pending for fp here>,
              "ts",
              "node_id", "ed25519":<node pub b64>, "sig":<b64>, "hops":4 }
```

Targets: the identity's home relay, if this relay knows it. When the home is known only because a sender named it (a standby onion terminal or a parked `send`, whose home may be the relay that was down), the pointer goes there **and** to the rendezvous set. Otherwise it goes to the identity's **rendezvous set**: the `k = 3` relays with the lowest rank among all verified peers plus this relay itself:

```
rank(node) = lowercase_hex( SHA-256( UTF8("r2r-rdv-v1" || fp || node_key) ) ) # compared as
strings
node_key   = node_id when known, else the peer's canonical address
```

The pointer is sent to each target other than this relay. Every relay computes the same list for the same peer view, so the wallet's own relay is likely to be among the relays it asks.

Receiving a pointer (relay sessions only; any problem means a silent drop):

1. The sending session is a **verified** relay (§11.2): a session that merely signed a `hello` may not deliver pointers. A pointer steers a wallet to a relay to gather mail, so it must not be plantable by anyone with a key.
2. `fp` is valid. `address` canonicalises and is not this relay. `ts` is not more than 600 s in the future and not older than the drop TTL.
3. `ed25519` decodes to 32 bytes, and `address` names a peer this relay has **verified** whose pinned `node_id` and `ed25519` equal the pointer's. A statement about a holder this relay has never reached, or signed with a key other than the one found there, is dropped. This is what stops a self-signed relay from pointing any fingerprint at an address of its choosing.
4. The signature verifies over the statement rebuilt with the **canonical** address.
5. The key `p:fp|address|ts` is new in the seen cache.
6. **Keep** the pointer if `fp` is homed here, is connected here, or this relay is in `fp`'s rendezvous set. Upsert it into `pointers` keyed `(fp, address)`, where a newer `ts` wins, with `expires_at = min(ts, now) + TTL`. Push `mail_at` to the identity's subscribed sessions.
7. Otherwise, if `hops > 0`, forward it with `hops - 1` to the home relay if known, else to the rendezvous set.

`locate` (§7.6) lists the kept pointers. After a successful collection from a holder, the collecting relay deletes that holder's pointer. A new pointer arrives if more mail lands there.

9.6 Safe deposit: collecting parked mail

The identity signs, **per holding relay**, a single-use instruction: "whoever presents this may collect my stored payloads here", plus "and you may delete them after handing them over" for `collect-delete`. The instruction names the holder but **not the collector**, so gathering mail never reveals where the identity lives.

Collector side (the identity's own relay, on `deposit`, §7.7): for each target, it canonicalises the address, skips itself, and sends over a relay link:

```
{ "t":"collect", "fp", "pubkey", "scope", "holder":"<canonical holder address>", "ts",
  "nonce", "sig", "want_max":200 }
```

It records a guard `collecting[fp|holder] = now + 600 s`.

Holder side (relay sessions only; any failure means a silent drop):

1. The sending session is a **verified** relay (§11.2). The authorisation is the identity's own and bound to this holder, but the link presenting it must belong to a relay this node has dialled and found, not to a session anyone could open with a key. A collect dropped here is retried: the wallet deposits again after every connect and every `mail_at` (§15).

2. `fp` is valid, `scope` is `collect` or `collect-delete`, and `nonce` is 16-64 characters.
3. `holder` **equals this relay's advertise address exactly**. An authorisation signed for another holder is worthless here.
4. `ts ≤ now + 600` and `ts ≥ now - 7 days`.
5. `fingerprint(pubkey) == fp`, and the signature over the `r2r-collect-v1` string (§7.7) verifies.
6. **Burn the nonce** (`used_authz`). A nonce that was already used is refused and logged as a replay.
7. Send up to 200 of the fingerprint's unexpired drops, oldest first, each as `{t:"collect_item", fp, id, body, created_at, hint?}`. Then send `{t:"collect_done", fp, address:self, count, more: count==200}`.
8. For `collect-delete`, delete exactly the handed-over ids in one transaction.

Collector on receipt: a `collect_item` is accepted only if the guard for `fp|<the sending link's peer address>` exists and has not expired. This stops a linked relay from stuffing arbitrary mailboxes. The item must also carry a valid UUID and a body of 1 byte to `max_payload`. It is stored through the normal path, which triggers push, `mail` and pointer emission. `collect_done` clears the guard and deletes the pointer for that holder. When `more:true`, the wallet issues a fresh authorisation for the rest.

9.7 Signalling and presence scope

`sig` is routed exactly like `send` (§9.2), but it is never stored: a local target that is offline yields `not_online`. Presence (`watch`) covers only the relay the watcher is connected to. To reach someone on another relay, a wallet either connects to that relay (the reference wallet keeps a connection to every relay used by any contact) or places the call and lets it fail.

10. Onion routing

10.1 Goal and guarantees

A sender who does not want relays to link sender and recipient wraps a payload in layers, one per **position** on a route. Each position names **1-3 candidate relays**, any one of which can open that layer, so a route survives relays that are down without the sender having to resend. The relay that receives a layer learns only:

- the socket the layer arrived on (its predecessor);
- the candidates for the next position, or, if it is a terminal, the recipient fingerprint and the recipient's relay.

It does **not** learn the sender, the recipient (unless it is a terminal), the route length, or its own position. The payload itself remains an end-to-end envelope (§5.7). Onion sealing protects **routing metadata**, not content.

10.2 The sealed box (one key, one relay)

The building block seals a short secret to one relay's X25519 key (§4.3):

```
eph_priv ← 32 random bytes;  eph_pub = X25519(eph_priv, 9)
shared   = X25519(eph_priv, recipient_x25519_pub)      # MUST reject all-zero (small-
order) results
key      = HKDF-SHA256(ikm = shared, salt = eph_pub || recipient_x25519_pub, info = "r2r-
seal-v1", L = 32)
nonce    ← 12 random bytes
aad      = 0x01 || eph_pub || recipient_x25519_pub
ct||tag  = AES-256-GCM-Encrypt(key, nonce, plaintext, aad) # tag is 16 bytes

sealed   = 0x01 || eph_pub[32] || nonce[12] || ct || tag[16]      # overhead: 61 bytes
```

To open it, a relay checks the version byte (0x01) and the minimum length of 61 bytes, derives the same key from its own private and public keys, and decrypts with the same AAD.

10.3 The layer: one content, several possible openers

Each onion layer is encrypted **once**, under a fresh 32-byte layer key K , and K is sealed (§10.2) to each candidate of that position:

```
K      ← 32 random bytes
slot_i = seal(candidate_i.x25519, K)          # 61 + 32 = 93 bytes each, i = 1..n, n
∈ {1,2,3}
head   = 0x02 || n || slot_1 || ... || slot_n
nonce  ← 12 random bytes
layer  = head || nonce || AES-256-GCM-Encrypt(K, nonce, plaintext, aad = head)
```

A relay opening a layer checks `0x02`, $1 \leq n \leq 3$ and the length, then tries **every** slot with its own key. The layer does not say which slot belongs to whom. It decrypts the content with the recovered K . Because the head is the AAD, slots cannot be swapped, stripped or re-ordered without breaking the tag. The minimum layer size is $2 + 93 + 12 + 16 = 123$ bytes.

10.4 Layer plaintexts

Plaintexts are binary. Each position therefore adds a fixed amount (about 300 bytes with three candidates), instead of inflating everything by base64 again.

```
routing layer  0x01 || n || n × ( len8 || "host:port" ) || inner layer
               -- the 1..3 candidates for the NEXT position, in the sender's preferred
               order,
               then the layer (§10.3) sealed to exactly those candidates

terminal layer 0x02 || len16be || meta || body
               meta = UTF-8 JSON {"to": "<fp>[<home relay>]", "id": "<uuid4>", "hint"?:
               "<≤128>"}
               body = the end-to-end envelope bytes (§5.7), 1..max_payload bytes
```

Validation at the relay:

- Routing: $1 \leq n \leq 3$, every address canonicalises (§3.4, default port 8787), and the inner layer is at least 123 bytes.

- Terminal: `meta` is a JSON object whose `to` parses like a `send` target (§9.1). `id` MUST be a UUID v4, and the relay generates one if it is missing. A `hint` longer than 128 characters is dropped.

10.5 Building a route (sender)

1. Choose the positions: an **entry** set, one or more **middle** sets, and a **terminal** set. Positions total 1–8, each with 1–3 candidates. The terminal set lists the recipient's relay first. Any further terminal candidates are standbys that will forward to it.
2. Obtain each relay's X25519 key from `GET /peers.json` (`node.x25519` for the relay itself, and `peers[].x25519` for its verified peers), from `GET /node.json`, or from `welcome`.
3. Build the terminal plaintext with `to = "<recipient fp>@<recipient's relay>"`, and seal it to the terminal set.
4. For each earlier position, from last to first: routing plaintext = the next position's candidates + the layer just built; seal it to this position's candidates.
5. Open a WebSocket to one of the entry candidates and send `{"t":"onion","blob":"<base64 layer>"}` with **no hello**. If that relay is unreachable, send the same blob to the next entry candidate: every entry candidate can open it.

10.6 Relay processing

```
on onion {blob, hops?}:
  blob must decode, 123 ≤ len ≤ max_frame           else err bad_field
  hops = frame.hops (default 8); if hops ≤ 0         → err hop_limit
  if "o:"+hex(SHA-256(blob))[0..32] was seen in the last 15 min → drop silently (loop
guard)
  plaintext = open the layer (§10.3)                  else err no_route ("not
addressed to us")
  layer = parse(plaintext)                            else err bad_frame
  if terminal: deliver (below)
  else:
    candidates = layer.next minus this relay itself   if none → err no_route
    forward { t:"onion", blob: base64(layer.inner), hops: hops-1 } to the candidates
(below)
  # success: send NOTHING back
```

Forwarding to candidates. The frame goes to the first candidate that already has a live link. Otherwise the relay dials the candidates in the sender's order, putting ones currently in dial backoff last. When a dial fails, the frame **moves to the next candidate at once**, instead of waiting in the 5-minute pending queue (§9.4). If every candidate fails, the layer is dropped.

Delivering a terminal layer. If `to` names no home relay, or names this relay, the payload is stored here like any dead drop (§8.2): pushed to live sessions, with a pointer emitted if the recipient lives elsewhere. If `to` names **another** relay, this relay is a standby. It forwards the payload there as an ordinary `send {id, to:"fp@home", body, hops}`. If home stays unreachable past the pending window, the payload is parked here, and pointers go to that home **and** to the recipient's rendezvous set (§9.5), so the recipient's wallet can gather it later.

Silence on success is required. An acknowledgement would tell the previous hop that the route ended here, which is exactly what the construction hides. Errors are not propagated backwards: each hop's error goes only to its own predecessor, so a sender hears only about the entry relay.

10.7 Limitations

- The **terminal relay learns the recipient fingerprint**, because the dead drop needs a name to file the payload under. A standby terminal also learns the recipient's relay.
- There is **no padding or cover traffic**. Timing is observable, and a party watching both ends of a route can try to correlate them. More positions make this harder only once the network has many relays.
- A layer that none of a position's candidates can take is lost, because it is sealed to them. The wallet's direct fallback (§10.8) covers this.
- A relay knows the candidate addresses for the next position, which is the price of alternatives. It still cannot tell which of them the route will actually use.

10.8 How the reference wallet sends through the onion

This is reference behaviour, not a wire rule. Every relay implements §10.3–§10.6, and the reference wallet uses them **by default** for every stored payload: texts, files, mail, group messages and delivery receipts. Call signalling (`sig`) stays on the direct path, because onion terminals only store.

Directory of hop keys. Two sources:

- the primary relay's `GET /peers.json`: its own entry plus verified peers **heard from within the last 3 minutes** (`last_ok`; relays ping their links every 30 s), cached for 3 minutes;
- every relay the wallet already talks to (`welcome.x25519`, `/node.json`).

Route shape. The Privacy setting chooses **Standard**, 3 positions (entry → middle → terminal), or **Extra privacy**, 5 positions (entry → 3 middles → terminal). With `r` usable relays besides the recipient's, each entry and middle set holds $\max(1, \min(3, \lfloor r \div (\text{middles} + 1) \rfloor))$ candidates:

1. **Terminal set:** the recipient's relay, then up to 2 standbys. Standbys are never taken from the previous set.
2. **Entry set:** relays the browser can reach, in random order, with the wallet's **primary last**: it already knows this device's identity from the authenticated connection.
3. **Middle sets:** random relays, preferring ones not yet used in this route and never ones from the set just before.

A small network produces shorter routes (fewer middles) before it produces thinner sets.

Submitting. The wallet builds a fresh route for every message and sends it on a new anonymous connection, trying the entry candidates in order. A relay that refuses the connection is skipped for 10 minutes. If no `err` arrives within 2.5 s, the message counts as **sent** (✓). If every entry was unreachable, the wallet builds one new route.

Delivery and fallback:

- The outbox keeps each message until the recipient's `dack` receipt arrives, which marks it **delivered** (✓✓).
- If no route can be built, sealing fails, the encoded blob would exceed about 900 KB, or the entry answers with an `err`, the message is sent at once on the direct path (§15.2).
- If no receipt arrives within **15 minutes**, it is resent once on the direct path, with the same wire id, so any copy that did arrive is deduplicated.

What each relay learns (Standard): the entry learns the device's IP address and the time, but not the identity or the recipient. The middle learns only relay addresses. The terminal learns the recipient and that the payload came from some middle relay.

Measured behaviour (reference relays and wallet, six local relays): 3-position routes with 2-3 candidates per position delivered in about 0.3 s, both ways, with every layer peeled by a relay. With two relays killed, messages kept arriving through their standbys. Within the 3-minute freshness window the dead relays dropped out of new routes. With the recipient's own relay down, a standby held the message, and it was delivered 10 s after the relay returned.

11. The peer-to-peer relay mesh

11.1 Bootstrap

A relay seeds its peer table from `peers.json` plus its **seed list**: the compiled-in bootstrap relays below, and any added with `--seed HOST:PORT` (repeatable) or

`R2R_SEEDS=host:port,host:port`. Seeds are dialled over plain `ws://` and are never pruned.

```
92.113.147.233:8787      (r2r.homes)
143.110.227.46:8787
164.90.207.73:8787
164.92.156.207:8787
165.22.204.117:8787
165.232.132.110:8787
```

Seed status comes **only** from this list. A `"seed"` flag stored in an older `peers.json` is ignored, so a relay removed from the network ages out of the table like any other unreachable peer.

Without `--advertise HOST[:PORT]`, a relay can still dial out, gossip and serve local clients, but peers cannot route to it.

11.2 Relay handshake

The dialling relay opens a WebSocket (path `/r2r`, subprotocol `r2r.v1`) and immediately sends its **identity frame** as `hello`. The accepting relay replies with its own identity frame as `welcome`. Both have the same shape:

```
{ "t":"hello" | "welcome", "proto":1, "role":"relay", "version":"1.0.0",
  "node_id":"<32 hex>", "advertise":"<host:port, may be empty>", "ed25519":"<b64>",
```

```
"x25519": "<b64>",
  "tls": <has a wss listener>, "ws_port": 8787, "wss_port": 8788 or 0,
  "ts": <unix>, "nonce": "<24 hex = 12 random bytes>", "sig": "<b64>" }
```

The signature covers:

```
"r2r-hello-v1\n" || node_id || "\n" || advertise || "\n" || decimal(ts) || "\n" || nonce
```

`advertise` is signed exactly as sent, before canonicalisation.

Verification by the receiving side. Any failure means `err not_authorized`, then close with code 1008:

1. If `node_id` equals this relay's own id, the dial looped back. Remember the dialled address as a self-alias, never dial or add it again, and close with code 1000.
2. `node_id`, `advertise`, `ed25519` and `sig` are present. `nonce` is 16–64 characters.
 $|\text{now} - \text{ts}| \leq 300 \text{ s}$.
3. `ed25519` decodes to 32 bytes, and `node_id == hex(SHA-256("r2r-node-v1" || ed25519)) [0..32]`.
4. The signature verifies.
5. `advertise` canonicalises (§3.4). `x25519`, if present, decodes to 32 bytes.
6. **Trust on first use (pinning):** if the peer table holds a **verified** entry for the dialled address or the advertise address whose `ed25519` differs, reject as impersonation ("node key does not match the pin"). Keys held on unverified entries are hints (§11.3) and are not pins: they never cause a rejection.
7. **Address policy:** `advertise` MUST be publicly routable. Loopback, RFC 1918, link-local, carrier-grade NAT, unspecified, multicast, documentation and reserved ranges (and their IPv6 equivalents, including IPv4-mapped forms) are rejected ("advertise address is not public"). The reference relay's `--allow-private-peers` switches this off for test benches only.

What a handshake proves. A valid signature proves that the peer holds the key behind `node_id`. It proves nothing about `advertise`: any key holder can name any address. An entry is therefore **verified** only when this relay dialled the address itself and the node answering there proved its key. Concretely, on success the relay:

- if the frame arrived on a link **this relay dialled**: marks the dialled address verified (`last_ok = now`, failures reset, `node_id`, `ed25519` and `x25519` pinned). If `advertise` differs from the dialled address, it is added as an unverified entry with the keys as hints. Any inbound relay session that had claimed the same advertise address with a different key is closed (code 1008): it was squatting on an address that now belongs to a proven key;
- if the frame arrived on an **inbound** link: adds `advertise` as an unverified entry (or leaves an existing one as it is) and records the keys as hints on it if they are empty. Nothing is marked verified. The peer manager will dial the address later (§11.4) and verify it then, whether or not the inbound link is still up.

In both cases the relay indexes the link under the advertise address, the dialled address and the node id, so frames for that address can use the live link, and:

- records the peer as a **plain ws:// peer at its advertise address**. `advertise` always names the plain port. `tls:true` only says the relay also has a `wss://` listener on `wss_port`, and MUST NOT make other relays dial TLS on the advertised port;
- if it received `hello:replies` `welcome`;
- **only on the first handshake of this connection**: sends its `peers` list and a `peers_req`. This prevents a repeated `hello` from being used as a cheap amplifier.

11.3 Gossip

```
{ "t":"peers_req" }
{ "t":"peers", "peers":[ { "address":"<host:port>", "tls":<bool>, "node_id":"...",
"ed25519":"...", "x25519":"..." } | "<host:port>" ] }
```

- A `peers` frame from a client session fails with `not_authorized`. From a relay session it is **acted on only if that session's address is verified** (§11.2): a relay whose advertise address this node has not yet dialled successfully has its lists silently ignored. A signed hello is enough to open a session, not enough to steer whom this relay connects to next. At most 128 entries are read.
- One session may **add at most 64 new addresses** to the table over its lifetime; further entries from it may still refresh key hints on addresses already known.
- An exported list contains **this relay first** (its own advertise address and keys), then up to 127 randomly shuffled entries that are verified or seeds. Unverified entries are never exported.
- **Merge rules**: skip entries whose `node_id` is this relay's own. Canonicalise and add new addresses; self, self-aliases and non-public addresses (§11.2 rule 7) are refused. Learn `node_id`, `ed25519` and `x25519` for an **unverified** entry only if the stored field is empty. **Never touch anything learned from a verified handshake**. Gossip can introduce addresses but never change a pin, and a hint it plants can never lock the genuine node out of its own address, since hints are not enforced.
- Every 60 s a relay sends `peers_req` to up to 3 of its relay links.

11.4 The peer table

Each entry holds: `host`, `port`, `tls`, `seed`, `verified`, `node_id`, `ed25519` (a hint until verified, the pin after), `x25519`, `first_seen`, `last_ok`, `last_attempt`, `failures`. There are at most 512 entries (`--max-peers`). When the table is full, the least useful non-seed entry is evicted, ranked by (unverified first, then most failures, then oldest `last_ok`).

- **Dial backoff** after `f` consecutive failures: `min(5 × 2(f-1), 300)` seconds after the last attempt.
- **Prune**: a non-seed peer with at least 5 failures whose last success (or first sighting) is more than 3600 s old is removed.
- **Link candidates** (to fill `--peer-dial-target`): unconnected peers past their backoff that are **verified or seeds**, ordered verified first, then fewest failures, then most recent `last_ok`.

- **Verification dials:** every maintain tick, at most **2** unverified non-seed entries past their backoff are dialled, longest-waiting first (`last_attempt`, then `first_seen`), regardless of whether they hold an inbound link. This is the only way an address learned from gossip or from an inbound hello is ever dialled, so a peer can make this relay open at most two connections per 15 s to addresses of its choosing, to public addresses only, with nothing observable returned. A `pong` refreshes `last_ok` only on a verified entry.
- **Address policy at dial time:** a hostname is resolved first and any resolved address that is not publicly routable is discarded; if none remains the dial fails.

peers.json on disk is rewritten atomically every 5 minutes and on shutdown:

```
{ "version":2, "updated_at":<unix>,
  "node":{"id","advertise","ed25519","x25519"},
  "peers":
  [ { "address","tls","seed","verified","node_id","ed25519","x25519","first_seen","last_ok","failures"} ] }
```

A hand-written file may be a plain array of `host:port` strings. Seeds are re-added on every load. `version 2` is the first in which `verified` carries the §11.2 meaning; a file with a lower version (or none) is loaded with every `verified` flag cleared, so each peer is re-verified by dialling. Non-public addresses in the file are dropped with a warning. **GET /peers.json** publishes only verified or seed entries: `{version, updated_at, node:{...}, peers:[{address, url, tls, verified, node_id, x25519, last_ok}], count}`.

11.5 Link maintenance timers (reference)

Timer	Period	Action
maintain	15 s	Dial link candidates up to <code>--peer-dial-target</code> (8) outbound links, then up to 2 verification dials (§11.4), and service pending queues (§9.4)
health	30 s	Send <code>{"t":"ping","nonce":<16 hex>,"time":<unix>}</code> to every relay link. A <code>pong</code> refreshes <code>last_ok</code>
gossip	60 s	<code>peers_req</code> to up to 3 links
persist	300 s	Prune, then rewrite <code>peers.json</code>
sweep	60 s, then 900 s	Storage expiry (§8.7)

Dial timeout is 15 s. Outbound links use user agent `r2r-relay/<version>`, SNI set to the peer host for TLS, TLS 1.2 or later, and **no certificate verification** unless `--peer-tls-verify`.

11.6 Why relays do not verify each other's certificates

Relays normally run self-signed certificates. Chain validation would either fail everywhere or force a certificate-authority dependency on every operator. TLS provides transport encryption, and the **Ed25519 pin** provides identity. For this purpose the pin is stronger than a certificate: it binds the node, not a hostname. Deployments that issue real certificates can enable `--peer-tls-verify`.

12. Registration and the invite tree

12.1 What registration is for

Relays are **invite-only by default**. A registration, made by claiming an invite code, does four things on the relay where it happens:

1. It records the identity's **home relay** in `identities`. From then on, a bare-fingerprint `send` or `sig` arriving at this relay is routed to that home (§9.2), and pointers for the identity are kept here (§9.5).
2. It records the identity's public key (and X25519 key), so the contact pages can resolve it (§12.8).
3. It places the identity in the relay's **account list**, which owners manage (§13).
4. It hands the new member **three invite codes** of their own, linked to the code they claimed. They start **locked** and unlock once the member has shown real activity (§12.5). This is how the network grows.

Each identity can register **once** per relay. A claim from a fingerprint that is already registered there is refused with `already_registered`, so to use a second code a person needs a new identity, and that identity has to earn its own codes.

A proved-but-unregistered key can still connect, send, fetch, keep a journal and upload blobs within the default allowance (1 MiB) and the common pool. Setup-card locators (§5.5) rely on this. The invite gate therefore controls **membership and growth**, meaning who is on the relay's books, who gets a home, and who can invite. Operators control **resource use** separately, through the default quota, per-user quotas and pools. An operator who wants usage to follow membership keeps the default quota small and raises it for members.

12.2 Invite code formats

Format	Example	Notes
Plain	R2R-28T5-CY4T-QKFF	3 groups of 4 Crockford-base32 characters = 60 bits of entropy
Hinted	R2R-5C71-93E9-28T5-CY4T-QKFF	2 extra leading hex groups encode the minting relay's IPv4 (5C.71.93.E9 = 92.113.147.233)
Legacy	3b241101-e2bb-4255-8caf-4136c566a962	UUID v4 from v1, stored lowercase

Generation: take 8 random bytes as a big-endian `u64` value `v`. For `i = 0..11`, character `i` is `ALPHABET[(v >> (55 - 5i)) & 31]`, with `ALPHABET = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"`. The code uses the top 60 bits. Insert `-` every 4 characters and prefix `R2R-`. If the relay's advertise host is a **literal IPv4 address**, insert its 8 uppercase hex digits as two groups after `R2R-`.

Normalisation (MUST be applied before any lookup):

1. Trim. If the input is 36 characters, it must be a UUID v4; lowercase it.

2. Otherwise uppercase it, require the `R2R-` prefix, and split the rest on `-` into groups of exactly 4 characters. There must be exactly 3 groups (plain) or 5 (hinted).
3. In **every** group, fold $I \rightarrow 1$, $L \rightarrow 1$ and $0 \rightarrow 0$.
4. Hint groups must be `[0-9A-F]`. Entropy groups must be Crockford (`[0-9A-Z]` minus `I L 0 U`), so `U` is rejected.
5. Rejoin with `-`.

A code survives handwriting, dictation and OCR.

12.3 Who can create codes: the invite "factories"

Codes exist only in the database of the relay that minted them. There is no network-wide registry. There are two kinds of source.

Unlimited sources, controlled by the people running the relay:

Source	How	issued_by	Contact token
Genesis	On the first start of an invite-only relay whose invite table is empty, 5 codes are minted and written to <code><data-dir>/genesis-invites.txt</code> (mode 0600). They are never logged .	<code>genesis</code>	no
Operator CLI	<code>r2r-relay --mint-invites N</code> (1-1000), printed to stdout	<code>operator</code>	no
Owner, from a wallet	<code>admin_invites {count: 1-50} (\$13)</code>	the owner's fingerprint	yes

Limited source, available to members:

Source	How	issued_by	parent_code	Contact token
Claim	Each successful claim mints exactly 3 new codes for the claimant, locked until the claimant is active (\$12.5)	the claimant's fingerprint	the claimed code	yes

Whoever sets up a relay is therefore its **invite factory**. They hold the genesis codes and can mint without limit, from the shell or, once they are an owner, from their phone. Codes from these sources are usable immediately. Every other member receives three codes, once, and those codes work only after the member has actually used the network. Invited people can invite others, but each new branch costs a new identity, real traffic and a vouch, and the relay's owners can cut off any branch.

A code minted by an identity (owner mints and claim children) also gets a random 16-byte **contact token** (32 hex characters). The page `/m/<token>` resolves it to the inviter's fingerprint and relay, so a new member can message the person who invited them straight away.

12.4 Claiming a code

The request is a `invite_claim` frame (§7.16) or `POST /invite/claim` (§16), carrying `code`, the §4.4 proof, and an optional `home_relay`. The relay:

1. Normalises the code. In invite-only mode an invalid code fails with `invite_invalid`.
2. **Verifies the proof.** This is mandatory: otherwise anyone could register their own relay as the home of someone else's fingerprint and have that person's mail routed to them.
3. Sets `home = canonical(home_relay)` if one was given (unparseable fails with `bad_field`), else this relay's advertise address.
4. In one `BEGIN IMMEDIATE` transaction:
 - Look up the code. If it is not found, the result is `invite_invalid`. If `revoked_at` is set, `invite_revoked`. If `burned_at` is set, `invite_used`. If it is still locked, `invite_locked`.
 - If `fp` is already in `identities`, the result is `already_registered`.
 - Burn it: `UPDATE invites SET burned_at=now, burned_by=fp WHERE code=? AND burned_at IS NULL AND revoked_at IS NULL AND locked=0`. If no row changed, a concurrent claimant won, and the result is `invite_used`.
 - Insert the identity: `home_relay = home, pubkey, registered_at = last_seen = now, received = 0`.
 - Insert 3 children: `issued_by = fp, parent_code = code, contact_token random, locked = 1`, hinted with this relay's IPv4 if it has one.
 - Commit. On any failure, roll back. **A failed claim mints nothing.**
5. Over WebSocket, store `x25519` if given. Reply `invite_ok` (§7.16), which names the **inviter** (the identity that issued the code, with its public key) when there is one, so the new member's wallet can greet them.

Burned and revoked codes are kept forever. Each is a code and a timestamp with no personal data, so it can never be replayed.

Open registration (`--open-registration`): no code is needed. The proof is still verified, an identity still registers only once, and the caller still receives 3 locked codes, which have no `parent_code`. The same client works against either kind of relay.

12.5 Activation: unlocking a member's codes

A member's codes unlock when the relay sees two things:

1. **Traffic.** The member has received and acknowledged at least **3 payloads** on this relay since registering (`identities.received`, increased on every `ack`). Only the key holder can acknowledge, so the count reflects messages the member's wallet actually fetched.
2. **A vouch from the inviter**, when an identity issued the member's code (claim children and owner-minted codes). The inviter's wallet sends `vouch {id: <member fp>}` (§7.18) after a real conversation, meaning at least one message each way. The relay accepts a vouch only from the identity whose code the member claimed. Members

who joined with a genesis, operator or open-registration code have no inviting identity, so traffic alone unlocks them.

As soon as both hold, the relay sets `locked = 0` on every code the member issued. `invite_status` (§7.19) reports the progress at any time. Relays cannot read messages, so the vouch is the inviter's word that the conversation happened. A person creating sock-puppet members still has to run every puppet as a separate identity that exchanges real messages. Each extra branch of the tree therefore costs real traffic, and every branch stays revocable (§12.7).

Locked codes are also refused by the web redemption page and reported as `"locked": true` by `GET /invite/check/<code>`.

12.6 Checking a code

`GET /invite/check/<code>` returns `{"ok":true,"open":<bool>,"locked":<bool>}` without burning the code. `open` is true only for a code that can be claimed right now. An invalid code gets HTTP 400 with `invite_invalid`.

12.7 Revocation and the cascade

`admin_revoke_invite {code, cascade}` (§13):

1. The named code is marked `revoked_at/by` if it is still **open**.
2. With `cascade` (the default in the relay), the relay walks the lineage **breadth-first over parent_code**. Every descendant is visited, including descendants reached through claimed codes. Every descendant that is still open is revoked.
3. Claimed codes, and the identities created from them, are **never** touched. Revocation stops the tree from growing further; it does not unregister anyone.

The reply reports `revoked` (0 or 1, for the named code) and `cascaded` (the number of open descendants revoked). Claiming a revoked code fails with `invite_revoked` (HTTP 410).

12.8 Web redemption, setup cards and contact pages

Relays built with the doorway module also serve a public web site with an invite flow for people who do not have a wallet yet:

Route	Behaviour
<code>GET /R2R-XXXX-...,</code> <code>GET /i/<code></code>	Short URL, the form QR codes encode. Redirects (302) to <code><base>/redeem?code=<canonical></code> . If the code is unknown here but hinted with another IPv4, it redirects to <code>https://<that IPv4>/redeem?code=...</code>
<code>GET /redeem?code=</code>	Preview: shows the code as valid, unknown, revoked, claimed, or not yet active (a member's locked code, §12.5). Does not burn it.
<code>POST /redeem</code> (form field <code>code</code>)	Burns the code for a web redemption. In one transaction it sets <code>burned_by = "card:" + first 16 hex of SHA-256(cardKey)</code> and stores <code>cards(card_key_hash = SHA-256(cardKey) hex)</code> . It then displays a setup card <code>R2RSC1:<fresh random cardKey hex>:<relay ws URL></code> (§5.5), plus the inviter's contact link if the code has a token. If the code was consumed between preview and POST, the page reports a race. No child codes are minted by web redemption.

Route	Behaviour
GET /card? code=	{code, redeem_url, host, contact_url or null}, for the Forge and wallets.
GET /m/ <token>	Contact page showing the inviter's contact link <code>r2r:<R2R_address>?relay=<url-encoded ws URL></code> (§4.2), ready to paste into a wallet, plus the relay URL.
GET /api/ contact/ <token>	{address: <inviter R2R_address>, pubkey: <inviter fingerprint>, ed25519: <inviter pubkey b64>, relay_url, revoked}, with fields emptied if the code was revoked. 404 if the token is unknown.
GET /api/ stats	{identities, invites_unused, invites_claimed}

The relay stores only the **hash** of a card key, never the key itself. A person onboarded by card gets their identity from the locator mechanism (§5.5). They are not registered in `identities` until they claim a code from their wallet.

12.9 Wallet onboarding paths (reference)

1. **With an invite code:** generate a random seed and create the vault (PIN of at least 8 characters). Then `invite_claim` on the relay named by the code's URL (`https://<host>[/path]/<code>` becomes `wss://<host>[/path]`), or by the IPv4 hint (`ws://a.b.c.d:8787`), or on a relay the user enters. `home_relay` is set to the primary relay's advertise address. Store the 3 returned codes (shown as locked), and make the relay the primary. If `invite_ok` names an inviter, add them as the first contact, and send them a `vouchreq` payload (§15.11) so their wallet can vouch once the two have chatted.
2. **With a setup card:** §5.5.
3. **Manual:** paste a key or generate one, and enter a relay URL. The identity is unregistered and works within the default allowance.

13. Relay ownership and administration

13.1 Owners

An **owner** is an ordinary identity listed in the relay's `owners` table. There is no admin password. Owner actions reuse the proof every wallet already sends on connect. The operator adds owners on the host:

```
r2r-relay --owner-add <64-hex fingerprint>
r2r-relay --owner-remove <fingerprint>
r2r-relay --list-owners
```

`welcome.owner` tells a wallet whether to show the owner panel. Every `admin_*` frame requires a proved session whose fingerprint is an owner. A non-owner receives `err not_authorized`, "not an owner of this relay", with exactly the same text whether or not the fingerprint exists. Clients cannot probe who the owners are.

13.2 Owner frames

Request	Reply
<code>{t:"admin_accounts", max?:1-500 (100), offset?:0}</code>	<code>{t:"admin_accounts", accounts:[Account], offset, default_quota_bytes}</code>
<code>{t:"admin_search_accounts", q:<1-128 chars>, max?:1-200 (50)}</code>	<code>{t:"admin_search_accounts", accounts:[Account], q}</code>
<code>{t:"admin_set_quota", id:<fp>, mb:<0-1048576> null}</code>	<code>{t:"admin_ok", action:"set_quota", id, quota_bytes, used_bytes, custom}</code>
<code>{t:"admin_set_ttl", id:<fp>, days:-1 1-36500 null}</code>	<code>{t:"admin_ok", action:"set_ttl", id, ttl_days}</code>
<code>{t:"admin_delete_account", id:<fp>}</code>	<code>{t:"admin_ok", action:"delete_account", id, drops_removed, journal_removed, blobs_removed}</code>
<code>{t:"admin_invites", count?:1-50 (3)}</code>	<code>{t:"admin_invites", invites:[code...]}</code>
<code>{t:"admin_list_invites", state?:"open" "locked" "burned" "revoked" "all", max?, offset?}</code>	<code>{t:"admin_list_invites", invites:[Invite], offset, state}</code>
<code>{t:"admin_revoke_invite", code, cascade?:true}</code>	<code>{t:"admin_ok", action:"revoke_invite", code, revoked, cascaded}</code>
<code>{t:"admin_set_advertise", host}</code>	<code>{t:"admin_ok", action:"set_advertise", advertise}</code>
<code>{t:"admin_list_rentals"}</code>	<code>{t:"admin_ok", action:"list_rentals", rentals:[...], committed_bytes, pool_bytes}</code>
<code>{t:"admin_list_vouchers"}</code>	<code>{t:"admin_ok", action:"list_vouchers", vouchers:[{payment_key,payout,cumulative_micro,sig,created_at,cumulative_total_micro, payout, vault, chain_id}</code>

Account = {id, home_relay, registered_at, last_seen, used_bytes, quota_bytes, custom_quota, pending, online}. Invite = {code, created_at, issued_by, state, has_contact_token, parent_code?, burned_at?, burned_by?, revoked_at?, revoked_by?}.

13.3 Host-side administration

These commands open the same SQLite database the running relay uses. WAL mode makes that safe, so no restart is needed:

```
r2r-relay --set-quota <fp> <MB>      r2r-relay --clear-quota <fp>      r2r-relay --list-quotas
r2r-relay --mint-invites <N>
```

Builds with the admin UI serve `/admin`, a browser console that signs the same owner frames with an Ed25519 key held in the page. It requires a secure context (https or localhost).

14. The storage market and on-chain settlement

14.1 Principles

1. **The user chooses the relays.** A wallet rents replica space on 1–5 relays the user picks. Storage is never assigned at random.
2. **Everything stored is ciphertext.** A malicious relay can delete data but never read it. The market trades availability.
3. **Payment follows performance.** Funds are prepaid into a contract the user controls, and a relay collects month by month, only after the wallet has audited it.
4. **No chain keys on the server.** A relay stores signed vouchers as opaque strings. The operator settles them from their own EVM wallet. A fully compromised relay yields no funds.
5. **The only address a user ever pays is the vault contract compiled into the wallet.** It is never taken from a web page or a relay.

14.2 Parameters

Parameter	Value
Chain	Any EVM chain. The deployment target is Base (chain id 8453)
Token	USDC (6 decimals). All amounts are in micro-USDC
Epoch	31 days (<code>--rent-epoch-days</code>)
First-epoch grace	35 days (<code>--rent-grace-days</code>)
Default price	500 000 micro-USDC per GiB-epoch (\$0.50/GiB-month; <code>--market-price</code>)
Minimum rental	1 MiB
Prepay cap	3 epochs ahead
Relay bond	10 USDC. Exit notice 60 days. Deposit withdrawal delay 40 days

The market is **enabled** only when `--pool-market-mb > 0` **and** `--payout-address` is set. `--vault-address` and `--chain-id` are published for wallets to check.

14.3 Discovery and renting

A relay that sells storage adds `market` to `/status.json`: `{enabled:true, price_gb_epoch_micro, epoch_days, pool_bytes, committed_bytes, available_bytes, payout, vault, chain_id}`. It adds `market:{price_gb_epoch_micro, available_bytes}` to `welcome`. A wallet builds its directory from its own relay's `/status.json` plus the `/status.json` of the peers listed in `/peers.json`.

```
{ "t":"rent", "bytes":<= 1048576, ≤ pool>, "payment_key":"0x<40 hex>" }
→ { "t":"rent_ok", "rental":<"<uuid>", "bytes", "price_epoch_micro", "epoch_days",
    "paid_until":<unix>,
    "payout":"0x...", "vault":"0x...", "chain_id" }
```

This requires identity. The relay refuses with `market_off` if the market is disabled. If `committed + bytes > pool` it refuses with `market_full`: the relay **never oversells**, because `committed` = the sum of active, unexpired rentals.

The epoch price is fixed when the rental is created, rounded up, and computed without overflow:

```
GiB = 1073741824
price_epoch_micro = (bytes div GiB) × price + ceil( (bytes mod GiB) × price / GiB )
```

The first epoch runs on trust: `paid_until = now + grace`. A renter who never pays costs the relay at most one epoch of one slice. The rented bytes add to the renter's allowance in every store path, and a renter is exempt from the common pool.

14.4 Vouchers

A voucher says: "I owe payee a **lifetime total** of cumulative micro-USDC." It is signed by the rental's payment key:

```
inner = Keccak-256( "r2r-voucher-v1" || vault_address[20] || uint256_be(chain_id)[32]
                  || payee_address[20] || uint256_be(cumulative_micro)[32] )
digest = Keccak-256( "\x19Ethereum Signed Message:\n32" || inner ) # EIP-191
personal_sign
sig = secp256k1 ECDSA over digest; RFC 6979 nonce; low-s; encoded as "0x" || hex(r[32] ||
s[32] || v[1]), v ∈ {0x1b, 0x1c}
```

The payment key's EVM address is `0x || hex( Keccak-256(X[32] || Y[32])[12..32] )`. Binding the vault address and chain id makes a voucher worthless on any other deployment.

Using a cumulative total makes vouchers **aggregating**: only the newest one matters, and redemption is replay-safe without nonces.

```
{ "t":"voucher", "rental":<"<uuid>", "payment_key":"0x...", "payout":"0x...",
  "cumulative_micro":<int>, "sig":"0x<130 hex>" }
→ { "t":"voucher_ok", "rental", "paid_until":<unix>, "cumulative_micro" }
```

The relay checks, in this order:

1. The session has an identity, the market is enabled, and the rental exists and belongs to this identity.

2. `payment_key` is `0x` plus 40 hex characters. `payout`, lowercased, equals this relay's payout address.
3. `payment_key` equals the key the rental was opened with (`rent`), `sig` is 132 characters starting with `0x`, and `cumulative_micro > 0`. 3a. **The signature is verified before anything is granted on its strength.** The relay rebuilds the digest above from its own `--vault-address`, `--chain-id` and payout address, recovers the secp256k1 signer exactly as the contract's `recover` does (65 bytes, `v < 27` means add 27, `s ≤ n/2`, non-zero result), and requires the recovered address to equal `payment_key`. Any other outcome is `not_authorized`. A relay with no `--vault-address` cannot build the digest and answers `market_off`. What the relay still cannot see is whether the payment key holds a deposit; that remains the contract's check at settlement, so an operator settles regularly and a rental whose vouchers fail to redeem lapses when its paid time runs out.
4. The voucher is stored verbatim if `cumulative` exceeds the highest one held for (`payment_key`, `payout`). Otherwise the relay replies `bad_field`, "must exceed the previous cumulative".
5. If `cumulative - previous < the rental's epoch price`, the voucher is **kept** (it is still the best money seen) but buys no time. The relay replies `bad_field`.
6. Otherwise it extends the rental: `paid_until = min( max(paid_until, now) + epoch, now + 3 × epoch )`, with `state = 'active'`.

A rental whose `paid_until` passes is marked `lapsed` by the sweep (§8.7).

14.5 Wallet obligations (reference policy)

- **One fresh payment key per rental**, from 32 random bytes reduced to `[1, n-1]`. It is never derived from the R2R identity, so on-chain observers see "some key paid relay Y" and never who.
- At most 5 active rentals. **Replica diversity**: never two replicas that share a payout address, an advertise host, or an IPv4 /16.
- **Replication**: copy the journal (opaque ciphertext) from the primary to each replica with `journal_append`, keeping a per-rental cursor. This runs after unlock, after journal flushes, and every 6 hours.
- **Audit, then pay**: when fewer than 7 days of paid time remain (or on demand), read a random sample of 3 journal entries back from the replica and check that at least one decrypts with the local journal key. **Only then** sign `cumulative = best_previous + price_epoch` and send the voucher. A failed audit means no payment, the replica is flagged, and the user should re-rent elsewhere.
- **Vault pinning**: the only vault address a wallet trusts for deposits is the one compiled into it. A relay whose `rent_ok.vault` differs is refused. The reference wallet's pinned address is empty until the contract is deployed, and it shows "do not deposit yet" until then.
- Message plaintext is rendered as text, never as HTML.

14.6 The R2RStorageVault contract

The contract is ownerless, non-upgradeable, and has no pause switch. There is one instance per chain.

State	Meaning
<code>relays[bytes32 nodeId] → {payout, registeredAt, exitAt}</code>	Relay registry. <code>registeredAt</code> gives a verifiable age
<code>registeredPayout[address] → count</code>	Only registered payout addresses can be paid
<code>balances[payer], withdrawAmount/After[payer]</code>	User deposits and pending withdrawals
<code>settled[payer][payee]</code>	Cumulative amount already redeemed
<code>lifetimeEarned[payee]</code>	Public reputation counter

Function	Rule
<code>register(nodeId)</code>	Pulls the 10 USDC bond and binds <code>msg.sender</code> as the payout address. A node id can be registered only once
<code>startExit(nodeId) / withdrawBond(nodeId)</code>	Bond refund only after the 60-day notice, during which wallets can see the exit and re-replicate
<code>deposit(amount)</code>	Credits the payer (the payment key)
<code>requestWithdraw(amount) / withdraw()</code>	Refund after 40 days. The amount is capped by the balance at withdrawal time, because redemptions continue during the window
<code>redeem(payee, cumulative, sig)</code>	Anyone may submit; the money always goes to <code>payee</code> . Requires <code>registeredPayout[payee] > 0</code> . Recovers <code>payer</code> from the sig (65 bytes; $v < 27$ means add 27; $s \leq n/2$; the recovered address must be non-zero). Pays <code>min(cumulative - settled, balance)</code> , which must be > 0 , then advances <code>settled</code> and <code>lifetimeEarned</code> . A partially paid voucher stays redeemable after a top-up
<code>redeemBatch(payee, cumulatives[], sigs[])</code>	Loops over <code>redeem</code>

Settlement flow: the owner fetches `admin_list_vouchers` and redeems the rows from their own browser wallet against the vault. The relay itself never holds chain keys.

Deployment status: the contract compiles and its ABI is in the repository, but it is **not yet deployed**. How the relay's 16-byte node id maps to the contract's `bytes32 nodeId` is also not fixed yet. It should be agreed before deployment; right-padding with zeros is the natural choice. Until then rentals run on the grace epoch.

15. Wallet behaviour: messaging, groups, mail, calls, sync

This section specifies what one wallet must understand from another, meaning the payload formats inside envelopes, and describes the reference wallet's algorithms. The algorithms are informative, but following them gives predictable behaviour.

15.1 Connections

- **The relay set** is the configured relay list plus every contact's relay. The first configured relay is the **primary**. It holds the journal, answers `locate`, runs owner functions, and is advertised to contacts as `rr/ra`.
- There is one persistent WebSocket per relay, at `<relayURL>/ws`. On open, send `hello` with the proof and `x25519`.
- **On welcome:** mark the relay live; fetch `/node.json` (to learn `node_id`, advertise and base); send `subscribe {push:true}`; watch all contacts' fingerprints; send own presence; flush the outbox; run the fetch loop. On the primary only: record owner, the usage figures and `max_payload`; GET `/status.json`; `locate`; the first time per identity, pull the journal (§15.8) and then flush it.
- **Fetch loop:** fetch `{since:0, max:100}`, repeated while `more`. It runs again on every mail frame.
- **Keep-alive:** `{"t":"ping"}` every 25 s. **Reconnect** after $\min(30\text{ s}, 1\text{ s} \times 1.7^n)$, where `n` counts consecutive failures.
- **Periodic tasks (30 s):** refresh stats and usage, reconnect missing relays, flush the journal.

15.2 Sending and routing

The payload is encrypted once per recipient (§5.7), queued in a persistent **outbox**, and flushed on every `welcome` and every 20 s. With onion routing on (the default), each entry is first sent through the onion as described in §10.8. The direct route below is used when onion routing is off, when it fails, or for the 15-minute fallback. The direct route for a contact:

```

if the contact's relay is connected:    send there, to = fp(contact)
elif the primary is connected:         send via the primary,
                                       to = fp@<contact.relayAddr or
advertise(contact.relay)> if the contact's relay ≠ primary
                                       to =
fp                                     otherwise
else:                                 keep queued; try to connect to the contact's relay

```

On the direct path, an outbox entry is removed when the relay answers `sent`, whatever the status. An onion-sent entry is removed when the recipient's `dack` arrives. A queued entry is always re-sent with the **same id**, so relays deduplicate it.

15.3 Delivery state

Tick	Meaning	Source
pending	Queued locally	wallet
sent (✓)	A relay accepted responsibility	the relay's <code>sent</code> frame, or an onion accepted by its entry relay without error
delivered (✓✓)	The recipient decrypted and stored it	the recipient's encrypted <code>dack</code> payload

Receipts are end-to-end: after processing a message, the recipient queues its wire id and, about once a second per sender, sends `{"t":"dack","ids":[<wire ids>]}` as an ordinary envelope. The relay has no receipt feature.

15.4 Receiving

On `drop` (pushed or fetched): queue the wire id for `ack` (batched every 250 ms, at most 200 ids). Open the envelope and discard it if it fails. Ignore duplicates, meaning an id seen among the thread's last 60 messages. Learn the sender's relay from `rr/ra`. Dispatch on the payload's `t`, persist, emit a delivery receipt, and journal the event.

15.5 Payload types (inside the envelope)

Every payload also carries `rr` (the sender's primary relay URL) and `ra` (its advertised `host:port`).

t	Fields	Notes
text	body	1:1 text
file	name, size, mime, vm?, dur? plus inline or blob, blobRelay	1:1 attachment. vm: "a" = voice message, "v" = video message; dur in seconds
mail	subj (≤200), body (rich text, ≤~40 KB), atts: [{name,size,mime,key,inline blob,blobRelay}] (≤8), to:[pubHex...] (≤8)	Multi-recipient mail. Each recipient gets its own envelope; to lists everyone
ginfo	gid, g:{gid,name,creator,members[]}	Group created or changed
gtext	gid, g, body	Group text
gfile	gid, g, name, size, mime, vm?, dur?, key plus inline or blob, blobRelay	Group attachment
dack	ids:[wire ids]	Delivery receipt

Unknown payload types MUST be ignored. Receivers MUST render all text as text; a mail body MUST be sanitised and never inserted as live HTML.

15.6 Attachments

Case	Encryption	Transport
1:1 file	nonce24 crypto_box(bytes, nonce, recipient_x25519, sender_x25519_secret) with no key field, so the receiver opens it with the sender's key	inline (base64) if the ciphertext is ≤ 48 KiB, else <code>POST /blob</code>
Group gfile, mail attachment	Random 32-byte key k: nonce24 crypto_secretbox(bytes, nonce, k), with key = hex(k) sent inside each envelope	Same

Blobs go to the relay the recipient uses, or the sender's primary, and `blobRelay` tells the recipient where to `GET /blob/<id>`. The wallet caps files at 16 MiB (the relay allows 24 MiB).

15.7 Groups and mail

Groups are **pure client-side fan-out**: the relay never learns a group exists. A group has a UUID `gid`, a name, a creator and at most **8 members** including the creator. Each group message is sent as a separate envelope to each other member, with fresh wire ids and a shared local message id. A receiver accepts a group payload only if the **sender is in the carried member list**. It learns new groups from any member, but accepts name or membership changes **only from the creator**. Group calls are a full mesh of at most **4** participants, where the participant with the lexicographically lower public key sends the offer to each peer.

15.8 The journal: encrypted history and multi-device restore

Every state change is recorded as an event, encrypted with the journal key (§5.3), and appended to the primary relay's journal:

```
entry = sbbox( JSON(event), journalKey )      # base64, sent as journal_append.data
```

Event k	Fields	Meaning
msg	peer: pubHex "g:"+gid, rec:{id, dir, kind, body/..., ts, status...}	A message sent or received
contact	c:{pub, addr, name, relay}	Contact added
rename	pub, name	Contact renamed
delcontact	pub	Contact removed
group	g:{gid, name, creator, members}	Group state
gdel	gid	Group deleted
mailrec	rec:{id, dir, peer, peers, subj, body, atts, ts, read}	Mail stored
mailread / maildel	id	Mail read or deleted
call	l:{pub, mode, dir, missed, dur, route?, t}	Call log entry
name	name	Own display name

Snapshots strip transient states: `pending` and `uploading` become `sent`, and large inline data (over 350 KB of base64) is dropped. **Upload:** events queue locally (`jout`) and are appended one at a time, oldest first, advancing `jseq`. **Restore:** `journal_read` from `jseq`, following `more`. Decrypt each entry and apply it idempotently (deduplicate by message id, contact key and so on), skipping undecryptable entries. On a new device, signing in with the same key replays everything from sequence 0. Journal sync can be switched off in the wallet.

Journal entries are opaque to relays, so a wallet can **migrate** or **replicate** them verbatim to another relay (§5.5, §14.5).

15.9 Calls (WebRTC)

The relay carries only `sig` frames. Media flows peer-to-peer, or through TURN.

Signal payload t	Fields
offer	sdp, mode ("voice" "video"), callId, iceRestart?, renegotiate?, gid?, gname?, joined?
answer	sdp, callId, renegotiate?, gid?
cand	cand (RTCIceCandidate), callId
bye	(none)
gjoin	callId, gid, joined:[pubHex] (group mesh coordination)

Every signal is an end-to-end envelope (§5.7) sent as `sig.body` to `fp` or `fp@relay`. The call starts with STUN only. If ICE fails, the wallet fetches `ice` credentials from its primary, sets them, restarts ICE with a new offer (`iceRestart:true`), and tries once more. Ringing times out after 35 s outbound (40 s for group calls) and 45 s inbound.

15.10 Gathering mail from other relays

After every connect, and 1.5 s after a `mail_at` (debounced), the wallet sends `locate` to its primary. If any pointers come back, it sends a `deposit` with scope `collect-delete` and one fresh authorisation per pointer address (§9.6). The gathered mail then arrives as ordinary drop frames.

15.11 Invites: greeting the inviter and vouching

- **New member:** after a successful claim, the wallet adds the inviter named in `invite_ok` as a contact ("Invited me"), and queues an end-to-end payload `{"t":"vouchreq","relay":"<the relay URL the code was claimed on>"}` to them. Its own codes show as locked, with a line saying what is still missing. The wallet refreshes `invite_status` when it connects, and a few seconds after receiving mail while still locked.
- **Inviter:** on receiving `vouchreq` from contact X, the wallet remembers the request. As soon as its thread with X holds at least one incoming and one outgoing message, it opens an authenticated connection to the named relay and sends `vouch {id: fp(X)}`. If the relay answers `not_authorised` (X was not invited by this identity there), the request is dropped.

16. HTTP API reference

Request Host header. Doorway pages and the redeem flow print the host the visitor used. A relay MUST use that value only when it parses as `hostname[:port]` or `[v6]:port` (§3.4 character rules), MUST fall back to its advertise host otherwise, MUST HTML-escape it wherever it is rendered, and MUST escape `<`, `>`, `&` and `U+2028/U+2029` in any JSON it embeds inside a `<script>` element.

The same routes are served on both ports. With `--base-path /P`, routes are also reachable under `/P/...`: the prefix is stripped before routing. `/R2R-...` invite URLs never collide with a base path, because a dash follows `R2R` where a mount point would have a slash.

Headers on every response: Server: r2r-relay/<ver>, X-Content-Type-Options: nosniff, Referrer-Policy: no-referrer, X-Frame-Options: DENY, Cache-Control: no-store (unless a route overrides it). JSON routes add CORS headers (Access-Control-Allow-Origin: *, methods GET, POST, OPTIONS, header Content-Type, max-age 600). OPTIONS requests get 204.

Limits: request bodies are limited to 64 KiB, except POST /blob, which is limited to max_blob (24 MiB). An oversized request gets 413 {"ok":false,"error":"too_big"} and the connection closes. The header read timeout is 30 s, or 120 s for uploads.

Authentication: X-R2R-Auth: base64(JSON proof) (§4.4). A missing or invalid header gets 401 {"ok":false,"error":"not_authorized"}.

Method and path	Auth	Response
GET /	-	HTML status page. When the doorway site is installed, the site's home page is served here and this page moves to /status
GET /health	-	{ok:true, uptime, tls}
GET /node.json	-	{node_id, ed25519, x25519, advertise, proto:1, version, base, ws_port, wss_port}
GET /peers.json	-	\$11.4 public format
GET /status.json	-	{node_id, version, base, uptime, peers_known, peers_active, connections, clients, relay_links, tls_connections, frames_in, frames_out, frames_forwarded, onion_peeled, drops_stored, drop_bytes, drops_accepted, drops_rejected, identities, invites_open, invites_burned, invites_revoked, pools: {common_used_bytes, common_cap_bytes, market_cap_bytes, personal_cap_bytes}, ttl_days, market?}
GET /invite/check/<code>	-	{ok:true, open, locked}. 400 invite_invalid for a malformed code
POST /invite/claim	proof in the JSON body	Body {code, id, pubkey, ts, nonce, sig, home_relay?} returns 200 {ok:true, invites[3], home_relay, node_id}. Errors: 400 (bad_frame / invite_invalid / not_authorized / bad_field / internal), 403 invite_locked, 409 invite_used / already_registered, 410 invite_revoked. The success body also carries locked and, when there is one, inviter
POST /blob	header	Raw bytes as the body. 200 {ok, id, size, expires_at, duplicate}. Errors: 400 empty, 413 too_big {max_bytes}, 413 quota, 500 internal
GET /blob/<64-hex id>	header	200 with the bytes, Content-Type: application/octet-stream, X-Content-Sha256: <id>. Errors: 400 bad_id, 404 gone
GET /ice	header	{ok:true, iceServers, ttl}
GET /assets/	-	Needs --assets DIR. {ok, count, assets:[{name, size, modified, sha256, url}]} for regular files ≤ 64 MiB directly in DIR
GET /assets/<name>	-	The file, with X-Content-Sha256. The name must be a single safe path component

Method and path	Auth	Response
GET / robots.txt	-	Disallows everything (the doorway serves its own)
GET /admin, / admin/ nacl.js	-	Owner console (admin-UI builds only)
Doorway routes	-	§12.8, plus the static site pages

Unknown routes get 404 {"ok":false,"error":"not_found"}. Methods other than GET, HEAD, POST and OPTIONS get 405.

17. Constants and configuration

17.1 Protocol constants (normative unless noted)

Name	Value
Client proof clock skew	± 600 s
Relay hello clock skew	± 300 s
Proof nonce length	16-64 characters
Pointer ts window	$\leq \text{now} + 600$ s, $\geq \text{now} - \text{TTL}$
Collect authorisation window	$\leq \text{now} + 600$ s, $\geq \text{now} - 7$ days
Invites minted per claim	3 (locked until the member is active)
Activation threshold	3 payloads received and acked, plus the inviter's vouch when an identity issued the code
Genesis invites	5
Onion: positions per route, candidates per position	1-8, 1-3
Onion: sealed key slot, layer overhead	93 bytes, $30 + 93 \times \text{candidates bytes}$ (+ routing header)
Wallet KDF	scrypt N = 65536, r = 8, p = 1, 32-byte output
Forwarding hop budget (default)	8 (--max-hops , 1-32)
Pointer hop budget, rendezvous k	4, 3
Seen-cache TTL and capacity	900 s, 200 000
Max payload (send body, onion terminal body)	262 144 bytes (--max-payload)
Max frame	1 MiB, or $2 \times \text{max_payload} + 65\,536$ when --max-payload is set
Max journal entry, journal page, journal page bytes	640 KiB, 200, 2 MiB
Max blob	24 MiB (--max-blob)

Name	Value
<code>fetch.max</code>	default 64, max 128
<code>ack.ids</code> per frame	256
Drops per recipient	4096
Default allowance	1 MiB (<code>--default-quota-mb</code>)
Drop TTL	7 days (<code>--ttl-days</code> , 1-365)
Collect batch	200
Deposit targets	1-16
Watch list	512
Gossip entries per <code>peers</code> frame	128
New addresses one relay session may add by gossip	64
Verification dials per maintain tick	2
<code>hint</code>	$\leq$ 128 characters
Target string	$\leq$ 400 characters

17.2 Reference behaviour constants

Name	Value
Inbound rate limit	40 frames/s, burst 120; close after 200 consecutive strikes (code 1013)
Error rate limit	2/s, burst 6
Send queue	256 messages (close with 1013)
WebSocket idle timeout	180 s
Pending queue per peer	64 frames, 4 MiB, 300 s grace
Peer dial target, dial timeout	8, 15 s
Backoff	5 s doubling, capped at 300 s
Prune	$\geq$ 5 failures and $>$ 3600 s since the last success
Max peers	512
Timers	maintain 15 s, ping 30 s, gossip 60 s (fan-out 3), persist 300 s, sweep 900 s
<code>used_authz</code> retention	8 days
Free-tier memo refresh	30 s

17.3 Relay configuration

Every flag has an environment equivalent, usually set in `/etc/r2r/relay.env`.

Flag	Env	Default
<code>--bind</code>	<code>R2R_BIND</code>	<code>0.0.0.0</code>
<code>--ws-port</code> / <code>--wss-port</code>	<code>R2R_WS_PORT</code> / <code>R2R_WSS_PORT</code>	8787 / 8788

Flag	Env	Default
<code>--cert / --key</code>	<code>R2R_TLS_CERT / R2R_TLS_KEY</code>	<code>/etc/r2r/tls/{cert,key}.pem</code>
<code>--no-tls</code>	-	TLS on
<code>--data-dir, --peers, --db, --node-key, --blob-dir</code>	<code>R2R_DATA_DIR, R2R_PEERS_FILE, R2R_DB_FILE</code>	<code>/var/lib/r2r, <data>/peers.json, <data>/r2r.db, <data>/node.key, <data>/blobs</code>
<code>--advertise HOST[:PORT]</code>	<code>R2R_ADVERTISE</code>	none (not routable)
<code>--seed HOST:PORT</code> (repeatable)	<code>R2R_SEEDS</code> (comma-separated)	added to the compiled-in seed
<code>--public-ws-url</code>	<code>R2R_PUBLIC_WS_URL</code>	none
<code>--base-path</code>	<code>R2R_BASE_PATH</code>	none
<code>--ttl-days, --max-payload, --max-blob, --max-hops, --max-peers, --peer-dial-target, --threads</code>	-	7, 262144, 24 MiB, 8, 512, 8, CPU count
<code>--open-registration, --no-gossip, --peer-tls-verify</code>	-	off
<code>--allow-private-peers</code>	<code>R2R_ALLOW_PRIVATE_PEERS</code>	off (test benches only)
<code>--default-quota-mb</code>	<code>DEFAULT_QUOTA_MB</code>	1
<code>--pool-common-mb, --pool-market-mb, --pool-personal-mb</code>	<code>R2R_POOL_*_MB</code>	0
<code>--market-price, --payout-address, --vault-address, --chain-id, --rent-epoch-days, --rent-grace-days</code>	<code>R2R_MARKET_PRICE_MICRO, R2R_PAYOUT_ADDRESS, R2R_VAULT_ADDRESS, R2R_CHAIN_ID</code>	500000, -, -, 8453, 31, 35
<code>--stun, --turn, --turn-user, --turn-pass, --turn-secret, --turn-secret-file</code>	<code>TURN_URL, TURN_USER, TURN_PASS, TURN_SECRET, TURN_SECRET_FILE</code>	Google STUN, none
<code>--assets, --doorway</code>	<code>R2R_ASSETS_DIR, R2R_DOORWAY_DIR</code>	none, <data>/doorway
<code>--user, --log-level</code>	<code>R2R_LOG_LEVEL</code>	-, info

v1 compatibility variables that are still honoured: `PORT`, `DATA_DIR`, `QUEUE_TTL_DAYS` and `BLOB_TTL_DAYS`. When both TTL variables are set, the larger value wins.

18. Security and privacy analysis

18.1 What is protected, and how

Asset	Protection	Residual risk
Message content	NaCl <code>box</code> end to end. Relays hold only ciphertext	No forward secrecy: static keys mean that anyone who steals a seed can decrypt every envelope they captured earlier
Sender authenticity	The box opens only for the true sender's X25519 key	Relays can replay an old envelope under a fresh wire id. The reference payloads carry no sender-signed id or timestamp, so a recipient cannot tell. A future payload version should include both inside the box
Mailbox access	A proof of possession of the key behind the fingerprint	Proofs can be replayed within ± 10 min, but only by someone who can already produce them
Home-relay integrity	Registration requires a proof, so nobody can redirect someone else's mail	Home relays are local knowledge, not network-wide
Relay identity	Ed25519 node key, node id bound to the key, TOFU pin per address set only by a handshake on a link this relay dialled, pinned keys that gossip can never overwrite	The first contact with an address is unauthenticated (TOFU); over <code>ws://</code> , an active attacker on that path could be pinned first
Peer table	Anyone may open a relay session, but only addresses this relay has dialled and found are verified, published, gossiped or used for rendezvous; peer lists are accepted from verified relays only, capped per session; unverified claims get at most 2 verification dials per 15 s; non-public addresses are refused everywhere	A node that really runs a reachable relay is verified like any other and may then gossip; its entries are still dialled at the bounded rate, to public addresses only, and it learns nothing from the outcome
Pointers	Signed by the holder; accepted only from a verified relay session and only about a holder this relay has itself verified at that address	Any genuinely reachable relay may still claim to hold mail for any fingerprint, since any relay may legitimately receive a <code>send</code> for anyone. A wallet that gathers from a pointer reveals its fingerprint and source address to that relay; gathering through an onion route or a proxy is the wallet's mitigation. Holder addresses are cleartext.
Presence	<code>watch</code> answers only a session that proved an identity	A proved identity can still ask whether any fingerprint is online here; the answer covers this relay only
Proof replay	Every accepted client proof's nonce is remembered for longer than the clock window; a second presentation is refused	A proof captured on a plaintext <code>ws://</code> link and presented before the wallet's own use would still win the race; <code>wss://</code> is the answer to capture

Asset	Protection	Residual risk
Web pages	The <code>Host</code> header is used only when it parses as a <code>hostname[:port]</code> , is HTML-escaped where rendered, and JSON embedded in a script is escaped for that context	Behind a proxy or cache that ignores <code>Host</code> , the pages still reflect whatever hostname the proxy forwards
Mailbox filling	Per-recipient allowance and drop count; the owner can raise the allowance; junk is cleared by fetching and acknowledging	Anyone may leave payloads for any fingerprint, including through onion routes, so a targeted mailbox can be filled to its allowance by an anonymous sender. This is a consequence of unsolicited dead drops and of never recording sender addresses; it is bounded and reversible, and a capability-gated store would be a protocol change
Collection	Holder-bound, single-use nonce, 7-day window, collector not named	A holder learns that someone collected for a fingerprint
Money	No chain keys on relays; vouchers bound to vault, chain and payee and verified by the relay before any storage is extended; wallet pins the vault address	The relay cannot see deposits, so a validly signed voucher from an unfunded key still buys up to 3 epochs of storage until settlement fails; bound by <code>--pool-market-mb</code> and by settling often. The wallet is the auditor. Audits sample 3 entries, which is probabilistic
Metadata at rest	No IPs, last-seen rounded to the hour, <code>secure_delete</code> , a privacy contract on logging	Relays see recipient fingerprints, sizes, timing and journal activity

18.2 Metadata exposure

- **A relay storing mail** learns the recipient fingerprint and the payload size and time. A relay a user connects to learns which fingerprint is online, and from which IP address while the socket is open, though that address is never written down.
- `rr/ra` in every payload reveal the sender's relay to the recipient. This is by design, so replies can be routed.
- **Plaintext** `ws:// hops` (the bridge) expose recipient fingerprints and destinations to network observers. **CDN-fronted** `wss://` endpoints expose the same data to the CDN. Clients that care should connect directly to a relay's `wss://` port.
- **Onion routing** hides sender-recipient linkage from all hops except the timing-capable global observer. The terminal hop still learns the recipient. There is no padding or cover traffic; that belongs in clients.
- **Presence and watch lists** tell the relay whose presence a user is interested in.

18.3 Key-management risks

- **PIN strength.** Whoever copies a device's `localStorage`, or holds a setup card (which gives the card key and the relays to query), can guess PINs offline. The KDF is scrypt at 64 MiB per guess (§5.2), which holds even strong GPUs to thousands of guesses per second, and the PIN policy refuses the guessable cases (short, digits-only, repeated). What remains is the user's choice of secret: a phrase of 4–5 random words is out of

reach, while a common single word with substitutions falls within months. The wallet says so as the user types.

- **No brain keys.** Private keys are always random; a typed phrase only ever unlocks the vault, and never becomes the key itself (§5.2).
- **Payment keys** are stored with the rental in the encrypted local state. They are separate from the identity by design.

18.4 Abuse and resource control

- **Anonymous sending** means anyone can fill a mailbox up to the recipient's allowance, or 4096 payloads. Relays refuse more with `quota` and never evict.
- **Robustness requirement.** A relay faces untrusted input from anonymous sockets. A conforming implementation **MUST** validate the **type** of every field it reads. A string where a number is expected, or an array where an object is expected, is `bad_field` (or is ignored on relay links). Malformed input **MUST NOT** terminate the process or affect other sessions. The reference relay handles every frame and every HTTP request inside a guard: a type error becomes `err bad_field` or HTTP 400 `{"ok":false,"error":"bad_field"}`, and any other failure becomes `internal`. As a last line of defence, its worker threads log and survive any exception that escapes a handler.
- **Error storms** are prevented by the terminal-error rule and the error rate limit (§6.3). **Amplification** is prevented by gossiping only on the first handshake.

18.5 Out of scope

Traffic analysis by a global passive adversary; a malicious recipient (who can always disclose what they receive); a compromised endpoint device.

19. Conformance checklist

19.1 A relay **MUST**

- [] Derive `node_id`, the X25519 key and the fingerprint exactly as in §4 (checked against Appendix A).
- [] Accept WebSocket upgrades on any path; handle text frames only; answer `err` for bad input; never answer an `err`; ignore unsolicited reply frames.
- [] Verify client proofs (§4.4) before binding a mailbox, claiming an invite, or serving an authenticated HTTP route.
- [] Validate field types, and survive malformed input (§18.4).
- [] Deduplicate by message id; honour hop budgets; never forward an onion layer back to itself; stay silent after accepting an onion layer.
- [] Implement the sealed box (§10.2), multi-candidate layers (§10.3) and binary layer parsing (§10.4) byte for byte; forward to candidates in order, moving on when a dial fails; deliver standby terminals to the named home (§10.6).

- [] Sign identity frames (§11.2) and pointer statements (§9.5); verify peers' frames; mark an address verified only after a handshake on a link this relay dialled; pin keys then, and never let gossip or an inbound hello overwrite or enforce a pin.
- [] Act on `peers` lists from verified relay sessions only, cap new addresses per session at 64, dial unverified entries at most 2 per maintain tick, and refuse non-public addresses in every path that can name one (§11.2 rule 7, §11.4).
- [] Make invite burns atomic with registration and child minting (§12.4); register an identity only once; mint members' codes locked and unlock them only on activation (§12.5); keep burned and revoked codes forever; walk the revocation cascade through claimed codes (§12.7).
- [] Deliver drops in `seq` order with an exclusive cursor; delete on ack; expire on TTL.
- [] Burn collection nonces; accept `collect_item` only for collections it requested from that peer.
- [] Never persist or log client IP addresses.

19.2 A relay SHOULD

- [] Match the §17 limits and timers, publish `/node.json`, `/peers.json` and `/status.json`, support `push`, pointers and deposit, and park undeliverable `sends` locally.
- [] Accept pointers only from verified relay sessions and only about verified holders (§9.5 rules 0 and 2); answer `watch` only for sessions that proved an identity (§7.11).
- [] Verify every voucher's secp256k1 signature against the rental's payment key over the §14.4 digest before extending `paid_until`; refuse vouchers without a configured vault address.
- [] Remember accepted proof nonces for at least the clock window and refuse a second presentation (§4.4 rule 7); accept `collect` only from verified relay sessions (§9.6 rule 0).
- [] Never render the request `Host` header, or any other request value, into HTML or an inline script without validating and escaping it for that context (§16).

19.3 A wallet MUST

- [] Encrypt every user payload in the §5.7 envelope and authenticate senders by opening the box, never by trusting a relay's `from`.
- [] Derive every PIN/passphrase key with the §5.2 scrypt KDF, enforce the PIN policy, and never turn typed text into a private key.
- [] Use UUID v4 message ids, and reuse the same id when resending.
- [] Ack only after the message has been durably persisted, or deliberately discarded.
- [] Render received text as text.
- [] For interoperability with other wallets: implement the vault KDF (§5.2), the setup-card locator (§5.5), the journal key and event encoding (§5.3, §15.8), and the payload types (§15.5).
- [] Market: pin the vault address; use one payment key per rental; audit before paying; use the §14.4 digest exactly.

20. Interoperability notes on the reference implementation

These are behaviours of the reference code (relay 1.0.0, wallet of September 2026) that a compatible implementation should know about. Where there is a choice, the recommendation keeps you compatible with the reference and moves the network towards the intended semantics.

1. **welcome to a client carries "role":"relay"**. It describes the relay, not the client. Do not branch on it in a client.
2. **proto is 1** even though the generation is called "v2".
3. **invite_ok.home_relay** always reports the claiming relay's own advertise address, even when the claim asked for a different `home_relay`. The database stores the requested value. Recommendation: relays echo the stored home.
4. **Invite hint detection in the wallet**. The wallet treats a code as hinted when the first 8 characters after `R2R-` are hexadecimal. The relay decides by group count: 5 groups means hinted, 3 means plain. About 1 in 256 plain codes starts with 8 hex characters and would be misread. Recommendation: use the group count.
5. **Relay URLs and paths**. Wallets append `/ws` to a relay URL for the socket, and use the same URL as the base for HTTP (`/node.json`, `/status.json`, `/blob`). Relays ignore the WebSocket path. Store relay URLs **without** a trailing `/ws` (with the base path if the relay is mounted under one), or HTTP metadata requests will 404.
6. **Quota arithmetic differs per path**. The drop path counts only drop bytes against the allowance. The journal and blob paths count total usage (\$8.5).
7. **send and onion need no hello**. They are accepted as the very first frame.
8. **sig across relays** trusts the forwarding relay's `from`. Always authenticate from the envelope.
9. **POST /invite/claim** (HTTP) does not record `x25519`. Only the WebSocket `invite_claim` does.
10. **Wallet ack timing**. The reference wallet schedules the `ack` 250 ms after processing a drop, while its local-storage write is debounced to 350 ms. An implementation should make sure local persistence completes **before** it acks.
11. **Card-onboarded users are unregistered**. Web redemption burns a code and mints no children, and the resulting identity has no `identities` row until it claims a code. How such users obtain invites of their own is an open question.

Appendix A: Test vectors

Every vector below was computed independently, then checked against the reference code:

- the **relay-side** vectors (A.1-A.8) were computed in Python and checked by a C++ harness linked against the reference relay library (`libr2r_core.a`). The harness checked fingerprint and node-id derivation, byte-identical Ed25519 signatures,

unsealing of the Python-sealed onion layer by the relay's `NodeIdentity`, a two-hop build/peel round trip, and invite-code normalisation;

- the **wallet-side** vectors (A.9–A.13) were computed by running the wallet's own `nacl.js`, `sha3.js`, `eth.js` and the Ed25519→X25519 routine from `core.js` under Node. The voucher signature was then independently verified with Python's `cryptography` (`secp256k1`, low-s).

All inputs are fixed byte patterns, not secrets. Hex is lowercase unless shown otherwise.

A.1 User identity and proof

```
seed =
000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f      (bytes 0x00..0x1f)
ed25519_pub (b64) = A6EHv/P0EL4dcN0Y50vAmWfk1jCbPQ1fHdyGZBJVMbg=
fingerprint      = 06f080fd2509682e820cbdfd3b10fe45c39d6e16e89fb3e1c31ac71ae65f3b61
ts, nonce        = 1754331000, 00112233445566778899aabb
signed string    = "r2r-client-
v1\n06f080fd2509682e820cbdfd3b10fe45c39d6e16e89fb3e1c31ac71ae65f3b61\n1754331000\n0011223344
5566778899aabb"
sig (b64)       =
VNWhTQV1+deh6G5QBBNqbAE0qi5VhE6b2z3dmMgyv0sz2ew29XxMFKZM1N0el9NN7A100UpikQh4ldiNY+hrBg==
```

X-R2R-Auth header, which is base64 of this exact compact JSON:

```
{
  "id": "06f080fd2509682e820cbdfd3b10fe45c39d6e16e89fb3e1c31ac71ae65f3b61",
  "pubkey": "A6EHv/POEL4dcN0Y50vAmWfk1jCbpQ1fHdyGZBJVMbg=",
  "ts": 1754331000,
  "nonce": "00112233445566778899aabb",
  "sig": "VNWhTQV1+deh6G5QBBNqbAE0qi5VhE6b2z3dmMgyv0sz2eW29XxMFKZM1N0e19NN7A100UpikQh4ldiNY+hrBg=="
}
```

A.2 Relay identity

```
node seed = 202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f (bytes 0x20..0xf)
ed25519_pub (b64) = Kay64UG8yvCyLhqU000LxzYeUm0L/hLI15S8kyKWbdc=
node_id = e3331ed6dce0d2adba2dbd8e7605915f
x25519_priv = 1a23ab6fb70bf0e5ddc656e86da68e35b07a0db11d9c2cf602ce59f915560889
(HKDF-SHA256, salt empty, info "r2r-x25519-v1")
x25519_pub (b64) = L6fz1q0QoL4X9s/7sTmojjs3hm/nCH3U2f1f0ozaDWA=
```

A.3 Relay hello, pointer and collect signatures

All use `ts = 1754331000` and, where applicable, `nonce = 00112233445566778899aabb` and advertise `203.0.113.10:8787`.

```
hello string = "r2r-hello-  
v1\ne3331ed6dce0d2adba2dbd8e7605915f\n203.0.113.10:8787\n1754331000\n00112233445566778899aab  
b"  
hello sig = akKZzGzU902/jtD3kJg3vsFUqlilyPdtBV7AYr2/6Z3Icc8INsrft9/  
R+ArhJygwZUqNYuuSShkBcg248T67AA== (node key)  
pointer string = "r2r-pointer-
```

Sealed to the A.2 relay with a fixed ephemeral key and nonce. Real senders MUST use fresh random values. The plaintext here is arbitrary sample bytes. In onion v2 this primitive seals each 32-byte layer key into a slot (A.4b).

Check: the blob starts with `0x01`, followed by `eph_pub` and the nonce. The AAD is `0x01 || eph pub || recipient pub`. Opening it with the A.2 relay's private key yields the plaintext.

A.4b Multi-candidate onion layer (§10.3), terminal plaintext (§10.4)

[illegible]

```
(0x02 || len16 || {"to": "<A.1
fp>@203.0.113.10:8787", "id": "3b241101-..."} || "hello")
layer (b64, 359 bytes) = AgIBExEK+AQ+9V+cmAzKKp25x/
MwVA6riGTJ9FNnJmT9HIBAQAQEBQAQEXWj7PQQ1tUqe8Hn0L0M9rU0T/
nUlc2Kzh6Ahh2jE6gQHBzvBCTlp64Sxm4phMTI0Bze/
YeDqRtEZkDi4fLVmds15ISgBxvSGCs7YNCBLBDHACAgICAgICAgJk5wMFWGBFh+18Vwnd+3D0ECG7yM48BiHDSd2
7JmIDWbwLs3xbMv0YvnHn6UNaoglmZmZmZmZmZmZmZmY9vTkpv2hGa/
pLIDPBmMZSSVBZZw1Nl3BV7qdWEKwdcFNgtvLIF/zCJNQG/
o2FbRp537Ri2Jyasssma4UapTi0ktn22fXGsRCWBpe2ccpgaVNzzYu3VUzXDcIqsCBHWfYJBId59wvrmPYUzmHPT5cFu
pNZkwwGyVRwikViCccl++uSZpWAJgzvbl1dGNCirjTf+0PTeXbTJabr1FQc7kQ=
```

Both candidates open it with the reference relay library and parse it as a terminal layer whose home is `203.0.113.10:8787`. The wallet's JavaScript produced it. A three-position route (2 + 3 + 2 candidates) built by the wallet was likewise peeled position by position by the relay library, with every candidate opening its layer.

A.5 Invite codes

```
entropy bytes      = 0123456789abcdef
plain code         = R2R-28T5-CY4T-QKFF
IPv4 hint 92.113.147.233 = 5C7193E9
hinted code        = R2R-5C71-93E9-28T5-CY4T-QKFF
normalise("r2r-5c71-93e9-28t5-cy4t-qkff") = R2R-5C71-93E9-28T5-CY4T-QKFF
normalise("R2R-28T5-CY4T-QKF0")           = R2R-28T5-CY4T-QKF0      (0 folds to 0)
normalise("R2R-28T5-CY4T-QKFU")           = <invalid>                (U is not Crockford)
```

A.6 Rendezvous ranking

For fingerprint `06f080fd2509682e820cbdf3b10fe45c39d6e16e89fb3e1c31ac71ae65f3b61`:

node_id	rank = SHA-256("r2r-rdv-v1" fp node_id)
00112233445566778899aabbccddeeff	c7ebf9b6a3ea4179719d89ba2f2bd3781b08d6cd40fb0ded62f52a15685d1ebe
ffeeddccbbaa99887766554433221100	c91dca45760b80d7f10bccdb3dd7d022acc665a7bb8f0cf3fc09302b5977dbdd
e3331ed6dce0d2adba2dbd8e7605915f	ca3e597c9f32908dbb0b8f42d26e16cdb9bb72c33ba54b6c0bd4274a470734ab
0f0e0d0c0b0a09080706050403020100	f7d7d9ace66b733385e30790d7e65588d266ae63a0616c1bc9aae436e4b8564d

With $k = 3$ the rendezvous set is the first three rows.

A.7 TURN REST credentials

```
secret = 's3cret', now = 1754331000, ttl = 600, fp = A.1 fingerprint
username = 1754331600:46ea36526cd6
credential = n6Ev6DcmiWtZAsOB2mRENlubSuU=
```

A.8 Market epoch price (500 000 micro-USDC per GiB-epoch)

bytes	price_epoch_micro
1,048,576	489
104,857,600	48,829
1,073,741,824	500,000
1,610,612,736	750,000

A.9 Hash sanity checks

```
SHA3-256("") = a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b49fa82d80a4b80f8434a
Keccak-256("") = c5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470
```

A.10 Wallet identity (same seed as A.1)

```
ed25519_pub      = 03a107bff3ce10be1d70dd18e74bc09967e4d6309ba50d5f1ddc8664125531b8
checksum         = fdbd93 (SHA3-256(pub || "R2R")[0..3])
address          = R2R_A0QPP7T_ZYIL4HLQ_3UM00S6A_TFT6JVRQ_T0SQ2XY5_3SDGIESV_GG4P3PMT
fingerprint      = 06f080fd2509682e820cbdfd3b10fe45c39d6e16e89fb3e1c31ac71ae65f3b61
x25519_secret    = 3894eea49c580aef816935762be049559d6d1440dede12e6a125f1841fff8e6f
(cclamp(SHA-512(seed)[0..32]))
x25519_pub (b64) = RwhQHhIhFH1RaQJ+1iuPlhYHKQKw/fxFGmM1x3qxzygE= (equals the
Edwards→Montgomery map of ed25519_pub: true)
```

A.11 Message envelope

Sender = A.10. Recipient seed = bytes 0x40..0x5f. The nonce is fixed here only for the vector.

```
recipient ed25519_pub = 2543b92ff1095511476adc8369db6ddc933665a11978dda1404ee1066ca9559d
nonce (24)           = c8c9cacbcccdcecf0d1d2d3d4d5d6d7d8d9daddbcdcdededf
payload              = {"t":"text","body":"hi","rr":"wss://r2r.example/
ws","ra":"203.0.113.10:8787"}
body (b64, 149 bytes) = A6EHv/
P0EL4dcN0Y50vAmWfk1jCbpQ1fHdyGZBJVMbjIycrLzM30z9DR0tPU1dbX2Nna29zd3t8pkeEFgg/Iz74/
urNNODjVM2TwHStV0nn5z2xrAZeLJSckB0F2zuMal2/
t1bA07eNaphHSNnea+fCnZbCL0pUzAcT6lPYxLoN+V0y044rtjfaWLddC9XPtPG6NKCXY=
```

The body's first 32 bytes are the sender's Ed25519 public key. The recipient opens it with its own X25519 secret (recipient_opens = true).

A.12 Vault, derived keys and setup card

```
KDF                = scrypt, N = 65536, r = 8, p = 1, dkLen = 32
passphrase         = river stone lantern eight
vault salt         = a0a1a2a3a4a5a6a7a8a9aaabacadaeaf
KDF(pass, salt)    = fc095eb7eb92ccc28a88395075e39b8b1042167ea01c4e7df9c34e275b506683
(matches Node's crypto.scryptSync)
store key          = 297ba67412c3810ea78e4845337513a48b5d83ba87fb9fed38687c18c43c34d5
(A.1 seed)
journal key        = 1102883c3f7a7de9a0d175a6fc0bd06e6714a0cb170b4101b0957f6355f811b4
backup key         = 9591e2e557a3ed2a5b5dfe0d80c2dee36a24c94d2ea5dac582340c3b71a8168b
store localStorage = r2r1_data_03a107bff3ce10be

card key           = 101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f
K = KDF(pass, cardKey) = b0761fb5bfa994ff8ea4dd5a9674030f05c6c23c4754eb79281fc7edff305397
locator seed       = be939019c41fd0e1f8e33085e9954e7c410787a918e75d99d905a0e67d066f61
(SHA3-256(K || "r2r-locator"))
locator fingerprint = 7c7367a021a634e50d0b5107000bc723639129e0135da77f9c84a4cf3dade2a9
record key         = 1fef48a3d07544a4448df26746a4a890c1e26e772d760ab3ef2b8a2e625648ff
(SHA3-256(K || "r2r-idblob"))
```

The wallet's scrypt reproduces the RFC 7914 §12 test vectors.

A.13 Storage voucher

```
payment private key = 4c0883a69102937d6231471b5dbb6204fe5129617082792ae468d01a3f362318
payment key address = 0x2c7536e3605d9c16a7a3d7b1898e529396a65c23
vault, chain_id     = 0x1111111111111111111111111111111111111111111111111111111111111111, 8453
payout (payee)      = 0x2222222222222222222222222222222222222222222222222222222222222222
cumulative_micro     = 489
inner hash          = e4162b4e96a99274413055ab991501f0d55e2d4afa94471faf0e6b269966bac0
sig (EIP-191)       =
0x4db3024afb2fdce238d7b2823f59014464c614621c190f98861a38829f8508bb5a868b971f2f5da04310cf7fde
8b95c383664fb96420361654d77758ea000d901c
```

Appendix B: Annotated live session

This is a real transcript captured on 23 September 2026 by driving two isolated reference relays (1.0.0, `--no-tls`, gossip off) with a minimal client, using the Appendix A identities.

`>>` is a client-to-relay frame and `<<` a relay-to-client frame. Keys in relay output appear in alphabetical order because of the reference JSON encoder. Bodies are placeholders here, not real envelopes. The capture predates invite activation: a current `invite_ok` additionally carries `locked`, `unlock` and, when an identity issued the code, `inviter` (\$7.16).

invite_claim (alice on A). Alice (the A.1 identity) claims a genesis code on relay A (127.0.0.1:18787). The code is hinted `7F00-0001` = 127.0.0.1. She receives three child codes.

```
>> [alice→A] {"t":"invite_claim","code":"R2R-7F00-0001-A5VH-M3JG-
QZDZ","id":"06f080fd2509682e820cbdfd3b10fe45c39d6e16e89fb3e1c31ac71ae65f3b61","pubkey":"A6EH
v/
POEL4dcN0Y50vAmWfk1jCbpQ1fHdyGZBJVMbg=","ts":1790153951,"nonce":"fa415268716543a87fb1e71d","
sig":"PX0eltiXUqj+LE1+8MK8A69K8Cp0glD7LE6Vnn20d6rYBDVxqw/
dhF3CjTl87fNZS0ZfB4XSasXaiKmPqfeCw==" ,"x25519":"0J/sKnR088gnARp/fn8hiU/
ESKf4T603BLT1Mg1IzIo="}
<< [alice→A] {"home_relay":"127.0.0.1:18787","invites":["R2R-7F00-0001-TRAR-AFYW-
S48G","R2R-7F00-0001-2B88-285F-DQ23","R2R-7F00-0001-5XT9-QCVN-
RZ78"],"node_id":"7a142aa40255aea4e29aee0fa15ebd46","t":"invite_ok"}
```

invite_claim reuse → error. Claiming the same code again fails. Burned codes are kept forever.

```
>> [x→A] {"t":"invite_claim","code":"R2R-7F00-0001-A5VH-M3JG-
QZDZ","id":"06f080fd2509682e820cbdfd3b10fe45c39d6e16e89fb3e1c31ac71ae65f3b61","pubkey":"A6EH
v/
POEL4dcN0Y50vAmWfk1jCbpQ1fHdyGZBJVMbg=","ts":1790153951,"nonce":"3195f0da81cd5ba9fd1fd158","
sig":"T20fQLGhN+clnt59XKi3sZOXIVN4rpKqMpAAoiSWwjfsWDLc8HL00NsjsMli0yjoPC9k9QTVBmPDaqewdorDg
=="}
<< [x→A] {"code":"invite_used","msg":"invite could not be claimed","t":"err"}
```

bob claims on B. Bob (seed 0x40..0x5f, the A.11 recipient) registers on relay B (127.0.0.1:28787).

```
>> [bob→B] {"t":"invite_claim","code":"R2R-7F00-0001-T4XF-8346-
JNCX","id":"fc37eb867c5d3238b056e8c794836d4d868242d729bc12adfa2a79da21569c0a","pubkey":"JU05
L/
EJVRFHatyDadtt3JM2ZaEZeN2hQE7hBmvpVZ0=","ts":1790153951,"nonce":"95ffae0eedfcde363f86901d","
```

```
sig": "R+2ZqpAHYKYbPzw9gI36y2Hj0+tvbri/Ddh1Uk2TBnIW+pqAgegQAdsBw5VFE53FHGsKwvJwv9lYo/
K3lFAAAA==" }
<< [bob@B] {"home_relay": "127.0.0.1:28787", "invites": [ "R2R-7F00-0001-33JH-2383-
PST6", "R2R-7F00-0001-4K45-VVYA-TGMA", "R2R-7F00-0001-PRB4-
VAFN-0GBZ" ], "node_id": "4e7f173cbbf91ae1bef5408152a62a96", "t": "invite_ok" }
```

hello / welcome. Both authenticate. Note the signed relay identity block inside `welcome`, and the per-identity fields (`you`, `pending`, `quota_bytes`, `owner`, `push`).

```
>> [alice@A]
{"t": "hello", "id": "06f080fd2509682e820cbdfd3b10fe45c39d6e16e89fb3e1c31ac71ae65f3b61", "pubkey":
"A6EHv/
POEL4dcN0Y50vAmWfk1jCbqQ1fHdyGZBJVMbg=", "ts": 1790153951, "nonce": "41cd487c054073953a0e8cb0", "
sig": "8ZaXMYcMNRtyZIQK8EbcRQBR3oJfFgFWqrZzIpMfYhBV2M4vwRF8ZZxI2iMH0ZE2U+dPDVR9AXUcqzdlc03QBg
==" }
<< [alice@A]
{"advertise": "127.0.0.1:18787", "ed25519": "mpadzlgrv4mySSCpqh7KgX1JsyWS0zSVvj0nUaVsR0U=", "max
_payload": 262144, "node_id": "7a142aa40255aea4e29aee0fa15ebd46", "nonce": "cdd82ecb21065baac2a5e
2cf", "owner": false, "peers": 0, "pending": 0, "proto": 1, "push": true, "quota_bytes": 1048576, "role":
"relay", "sig": "u0jDtUbySKyKOSMYJewXoGK2VyzCUAjiTR+NfGcEyj+AdNzrECaFndA60zrYNssm8R14w3w8CKUtr
6DXeob7Bg==", "t": "welcome", "tls": false, "ts": 1790153951, "ttl_days": 7, "used_bytes": 0, "version":
"1.0.0", "ws_port": 18787, "wss_port": 0, "x25519": "QoXJ20tqzc7mP9k2qgdI9BYolAH1hko7pIyYsFpN8Vs=
", "you": "06f080fd2509682e820cbdfd3b10fe45c39d6e16e89fb3e1c31ac71ae65f3b61" }
>> [bob@B]
{"t": "hello", "id": "fc37eb867c5d3238b056e8c794836d4d868242d729bc12adfa2a79da21569c0a", "pubkey":
"JU05L/
EJVRfHatYDadt3JM2ZaEZeN2hQE7hBmYPVZ0=", "ts": 1790153951, "nonce": "04e8d17ed9a80138021a2d57", "
sig": "iAfAohQs/UXSJ9/
zoaMztSyXCEnuEvrhelixxIgRIJf8135atxoVq8CG7M9VzerRlGfYr8nak5urPF00lQV0Ag==" }
<< [bob@B] {"advertise": "127.0.0.1:28787", "ed25519": "LSiDxxdSxtwQs0HwwAtur8b8LLaQDzrn46/
ITfM7puo=", "max_payload": 262144, "node_id": "4e7f173cbbf91ae1bef5408152a62a96", "nonce": "29dfac
7ab4e14c4fe2905739", "owner": false, "peers": 0, "pending": 0, "proto": 1, "push": true, "quota_bytes":
1048576, "role": "relay", "sig": "/
6ug9Iz1UsWB3bKz4Xw70i9tJ+sCApRK1nAv9hZzP6Q7LsBWHzavehCm9NAzVD5iCJT0JhpHHu+aoASjuIEDAQ==", "t":
"welcome", "tls": false, "ts": 1790153951, "ttl_days": 7, "used_bytes": 0, "version": "1.0.0", "ws_por
t": 28787, "wss_port": 0, "x25519": "DJol/
QDozegnLMx7mus8jCY0hm+Hkof7rLoiLQR6nhg=", "you": "fc37eb867c5d3238b056e8c794836d4d868242d729bc
12adfa2a79da21569c0a" }
>> [bob@B] {"t": "subscribe", "push": true}
<< [bob@B] {"pending": 0, "t": "mail" }
```

send qualified fp@relay (forwarded A→B, pushed live to bob). Alice sends to `bob@B` through A. A dials B, completes the signed relay handshake, and forwards the frame. B stores it and pushes the `drop` live to Bob's `push:true` session, followed by `mail`.

```
>> [alice@A] {"t": "send", "id": "cadddd34-
f3a9-4ccb-9467-0c4251fc01a0", "to": "fc37eb867c5d3238b056e8c794836d4d868242d729bc12adfa2a79da2
1569c0a@127.0.0.1:28787", "body": "b3BhcXVlLWNpcGhlcnRleHQtMQ==" }
<< [alice@A] {"id": "cadddd34-
f3a9-4ccb-9467-0c4251fc01a0", "status": "forwarded", "t": "sent", "via": "127.0.0.1:28787" }
<< [bob@B]
{"body": "b3BhcXVlLWNpcGhlcnRleHQtMQ==", "created_at": 1790153951, "expires_at": 1790758751, "id":
"cadddd34-f3a9-4ccb-9467-0c4251fc01a0", "seq": 1, "t": "drop" }
<< [bob@B] {"pending": 1, "t": "mail" }
```

fetch + ack. The same payload is redelivered by `fetch` until it is acked. `seq` is the cursor.

```
>> [bob@B] {"t": "fetch", "since": 0, "max": 64}
<< [bob@B]
{"body": "b3BhcXVlLWNpcGhlcnRleHQtMQ==", "created_at": 1790153951, "expires_at": 1790758751, "id":
```

```
"cadddd34-f3a9-4ccb-9467-0c4251fc01a0","seq":1,"t":"drop"}
<< [bob@B] {"count":1,"cursor":1,"more":false,"t":"fetch_done"}
>> [bob@B] {"t":"ack","ids":["cadddd34-f3a9-4ccb-9467-0c4251fc01a0"]}
<< [bob@B] {"removed":1,"t":"ack_ok"}
```

bare fingerprint to A (A does not home bob) → stored on A, pointer to B, mail_at.

A bare fingerprint arrives at A, which does not know Bob's home, so A stores it and emits a signed pointer to Bob's rendezvous set, which includes B. B homes Bob, keeps the pointer, and nudges Bob with `mail_at`. `locate` lists it.

```
>> [alice@A] {"t":"send","id":"23cecb8-73b6-4de6-
abbd-65d7b09ad10a","to":"fc37eb867c5d3238b056e8c794836d4d868242d729bc12adfa2a79da21569c0a","
body":"b3BhcXVlLWNpcGhlcnRleHQtMg==" }
<< [alice@A] {"expires_at":1790758751,"id":"23cecb8-73b6-4de6-
abbd-65d7b09ad10a","status":"stored","t":"sent"}
<< [bob@B] {"address":"127.0.0.1:18787","count":1,"t":"mail_at"}
>> [bob@B] {"t":"locate"}
<< [bob@B] {"pending":0,"pointers":
[{"address":"127.0.0.1:18787","count":1,"ts":1790153951}], "t":"located"}
```

deposit (collect-delete) → gathered to B. Bob signs a holder-bound, single-use authorisation for A. B presents it to A, A hands over the payload and deletes it, and B stores it. It then reaches Bob as a pushed `drop`, which appears interleaved in the next block because frames are asynchronous.

```
>> [bob@B]
{"t":"deposit","id":"fc37eb867c5d3238b056e8c794836d4d868242d729bc12adfa2a79da21569c0a","pubk
ey":"JU05L/EJVRFHatyDadtt3JM2ZaEZeN2hQE7hBmypoVZ0=","scope":"collect-delete","targets":
[{"address":"127.0.0.1:18787","ts":1790153951,"nonce":"826024f4b6090c861dc1814f","sig":"jtYv
KwsGD70CEgEM5dbxq0aNaeyAY2I4RtiEPokTC37/FMEcTua0Nl0ziIVU7ARSIm5NdNLgZFaHPTji6RCDQ=="}}}
<< [bob@B] {"t":"deposit_ok","targets":1}
```

journal. Append and read back one opaque journal entry. The gathered `drop` and its `mail` arrive in between.

```
>> [bob@B] {"t":"journal_append","data":"ZW5jLWpvdXJuYWwtZW50cnk="}
<< [bob@B] {"seq":1,"t":"journal_ok"}
>> [bob@B] {"t":"journal_read","since":0,"max":200}
<< [bob@B]
{"body":"b3BhcXVlLWNpcGhlcnRleHQtMg==","created_at":1790153951,"expires_at":1790758751,"id":
"23cecb8-73b6-4de6-abbd-65d7b09ad10a","seq":1,"t":"drop"}
<< [bob@B] {"pending":1,"t":"mail"}
<< [bob@B] {"cursor":1,"entries":
[{"created_at":1790153951,"data":"ZW5jLWpvdXJuYWwtZW50cnk=","seq":1}], "more":false,"t":"jour
nal"}
```

quota / ice / watch / sig. Usage report; ICE servers (STUN only in this test); presence (Bob is offline as seen from relay A, because presence is per relay); `sig` to an offline local target yields `not_online`; `sig` to `bob@B` is forwarded, and B stamps `from`.

```
>> [bob@B] {"t":"quota"}
<< [bob@B]
{"custom":false,"max_payloads":4096,"quota_bytes":1048576,"t":"quota","used_bytes":36,"used_
payloads":1}
>> [bob@B] {"t":"ice"}
<< [bob@B] {"iceServers":[{"urls":"stun:stun.l.google.com:19302"}],"t":"ice","ttl":600}
>> [alice@A] {"t":"watch","ids":
```

```
[ "fc37eb867c5d3238b056e8c794836d4d868242d729bc12adfa2a79da21569c0a" ] }
<< [alice@A]
{ "id": "fc37eb867c5d3238b056e8c794836d4d868242d729bc12adfa2a79da21569c0a", "seen": 0, "state": "offline", "t": "presence" }
>> [bob@B]
{ "t": "sig", "to": "06f080fd2509682e820cbdfd3b10fe45c39d6e16e89fb3e1c31ac71ae65f3b61", "body": "c2Rw" }
<< [bob@B] { "code": "not_online", "msg": "that peer is not connected", "t": "err" }
>> [alice@A]
{ "t": "sig", "to": "fc37eb867c5d3238b056e8c794836d4d868242d729bc12adfa2a79da21569c0a@127.0.0.1:28787", "body": "ZW5jLW9mZmVy", "call": "c1" }
<< [bob@B]
{ "body": "ZW5jLW9mZmVy", "call": "c1", "from": "06f080fd2509682e820cbdfd3b10fe45c39d6e16e89fb3e1c31ac71ae65f3b61", "t": "sig" }
```

Onion route A → B. The reference probe sealed a two-layer route, A then B, and submitted it to A. A peeled its layer and forwarded the inner layer to B. B stored the terminal payload. Neither relay acknowledged anything, and Bob then fetched it:

```
$ r2r-probe --url ws://127.0.0.1:18787 --http http://127.0.0.1:28787 \
  --hops 127.0.0.1:18787,127.0.0.1:28787 onion <bob-fp> "via onion"
onion accepted; terminal payload id 32277858-5188-4f7e-9946-28b1a0ed610e

<< [bob@B] { "t": "drop", "id": "32277858-5188-4f7e-9946-28b1a0ed610e", "body": "<b64 of 'via onion'>", ... }
<< [bob@B] { "count": 2, "cursor": 2, "more": false, "t": "fetch_done" }
```

Appendix C: Glossary

Term	Meaning
Allowance	The bytes an identity may store on a relay: a custom quota or the default, plus active paid rentals
Bridge	Forwarding a frame that arrived over <code>wss://</code> on over <code>ws://</code> , because the next peer speaks only plaintext
Card key / pepper	The 32 random bytes on a setup card. With a PIN, it derives the locator
Collection authorisation	An identity-signed, holder-bound, single-use permission to take its parked mail
Dead drop	A stored payload waiting for its recipient (<code>drops</code> table)
Doorway	The optional public web site module of a relay (invite redemption, contact pages, status)
Envelope	<code>sender_pub nonce box</code> : the end-to-end encrypted unit between wallets
Fingerprint	<code>hex(SHA-256("r2r-id-v1" ed25519_pub))</code> , an identity's address on relays
Genesis codes	The first 5 invite codes a new invite-only relay mints for its operator
Home relay	Where a registered identity's mail should be delivered; known only to the relay where it registered
Activation	The point at which a member's own invite codes unlock: 3 payloads received and acked, plus the inviter's vouch when an identity invited them

Term	Meaning
Invite factory	Informal name for the unlimited code sources (genesis, operator CLI, owner minting), as opposed to members' three locked codes
Journal	A per-identity, append-only log of encrypted events on the relay; the basis of history sync and restore
Locator	The identity derived from a card key and a PIN, whose journal stores the real identity's seed
Node id	32 hex characters of <code>SHA-256("r2r-node-v1" node_ed25519_pub)</code> ; a relay's identity
Owner	An identity allowed to administer a relay through signed frames
Pin (TOFU)	The Ed25519 key first seen for a peer address; later handshakes must match it
Pointer	A relay-signed statement "I hold N payloads for fingerprint F"
Primary relay	The first relay in a wallet's list: journal, owner panel, locate
Rendezvous set	The 3 relays whose SHA-256 rank is lowest for a fingerprint; where pointers go when the home is unknown
Seen cache	A 15-minute memory of message, onion and pointer keys, for deduplication and loop prevention
Setup card	<code>R2RSC1:<cardKey>:<relays></code> , a printed QR that restores or creates an identity with a PIN
Voucher	An EIP-191 signed statement of a cumulative amount owed to a relay's payout address
Wallet	The R2R client application; holds identity and payment keys

End of specification.